Ferramentas e Aplicações Multimédia

Sebenta da Unidade Curricular

João Pavão Sérgio Madeira

2026-04-03

Índice

Prefácio **1**
Organização da sebenta . . . 1
Como usar esta sebenta . . . 1
Autores . . . 1

1 Multimédia Digital — Fundamentos e Organização da Sebenta **2**
Introdução . . . 2
1.1 O que é Multimédia? . . . 2
1.1.1 Definição e integração de media . . . 2
1.1.2 Multimédia analógica vs digital . . . 2
1.2 Tudo é Representação Numérica . . . 3
1.2.1 O princípio fundamental . . . 3
1.2.2 Sinal analógico vs sinal digital . . . 3
1.2.3 O que é um ficheiro? . . . 4
1.3 A Web como Sistema Multimédia . . . 4
1.3.1 Arquitectura da web . . . 4
1.3.2 A performance como parte do design . . . 5
1.4 Estrutura da UC e da Sebenta . . . 5
1.4.1 Programa e blocos temáticos . . . 5
1.4.2 Avaliação . . . 5
1.5 Conteúdos Disponíveis na Sebenta . . . 6
Capítulo 2 — Sistemas Numéricos e Codificação . . . 6
Capítulo 3 — Digitalização e Compressão de Som e Imagem . . . 6
Capítulo 4 — Imagem Digital e Cor para Web . . . 6
1.6 Síntese do Capítulo . . . 7

2 Sistemas Numéricos e Codificação **8**
2.1 Motivação . . . 8
2.2 O Sistema Decimal . . . 8
2.2.1 Representação posicional . . . 8
2.3 O Sistema Binário . . . 9
2.3.1 Por que base 2? . . . 9
2.3.2 Estrutura do sistema binário . . . 9
2.3.3 Conversão binário → decimal . . . 9
2.3.4 Conversão decimal → binário . . . 10
2.4 O Sistema Hexadecimal . . . 10
2.4.1 Motivação e estrutura . . . 10
2.4.2 Relação directa com o binário . . . 11
2.4.3 Conversão decimal → hexadecimal (método directo) . . . 11
2.4.4 Conversão decimal → hexadecimal (método prático) . . . 11
2.4.5 Conversão hexadecimal → decimal (método directo) . . . 12

2.4.6 Conversão hexadecimal → decimal (método prático) 12
2.4.7 Conversão binário → hexadecimal 12
2.4.8 O binário como ponte entre bases 12
2.5 Bits e Bytes 13
2.5.1 Definições 13
2.5.2 Capacidade de representação 13
2.6 Aplicação em Multimédia 13
2.6.1 Representação de texto — ASCII e Unicode 13
2.6.2 Representação de cor — modelo RGB 14
2.6.3 Representação de imagem, áudio e vídeo 14
2.7 Síntese 14
2.8 Exercícios 15
2.8.1 Conversões numéricas 15
2.8.2 Aplicação em multimédia 15

3 Digitalização e Compressão de Som e Imagem 17
3.1 Enquadramento 17
3.2 Sinais no Mundo Real 17
3.2.1 Sinais contínuos e sinais discretos 17
3.3 Digitalização 18
3.3.1 O processo de digitalização 18
3.4 Amostragem 18
3.4.1 O que é a amostragem 18
3.4.2 Frequência de amostragem 18
3.4.3 Teorema de Nyquist 19
3.5 Quantização 19
3.5.1 O que é a quantização 19
3.5.2 Profundidade de bits 21
3.6 Compressão 21
3.6.1 A necessidade de compressão 21
3.6.2 Compressão sem perdas 22
3.6.3 Compressão com perdas 23
3.6.4 O compromisso qualidade vs. tamanho 24
3.7 Formatos de Imagem 24
3.7.1 JPEG 24
3.7.2 PNG 24
3.7.3 SVG 25
3.7.4 WebP 25
3.8 Som Digital 25
3.8.1 Representação do som digital 25
3.8.2 Formatos áudio 26
3.9 Síntese 26
3.10 Exercícios 26
3.10.1 Digitalização e amostragem 26
3.10.2 Compressão e formatos 27

4 Imagem Digital e Cor para Web 28
Introdução 28
4.1 A Imagem Digital 28
4.1.1 Definição e estrutura de uma imagem rasterizada 28
4.1.2 Tamanho de um ficheiro bitmap 28
4.1.3 Imagem rasterizada vs imagem vetorial 29

4.1.4 Formatos de imagem bitmap para web . . . 30
4.1.5 Imagens vetoriais — SVG . . . 30
4.1.6 Entrelaçamento (Progressive Rendering) . . . 31
4.2 Luz e Percepção da Cor . . . 31
4.2.1 O espectro electromagnético e a luz visível . . . 31
4.2.2 Anatomia da retina e fotorreceptores . . . 32
4.2.3 Sensibilidade espectral dos cones S, M, L . . . 32
4.2.4 Absorção relativa e construção da percepção cromática . . . 32
4.2.5 Síntese aditiva e síntese subtrativa . . . 32
4.3 Modelos de Cor . . . 33
4.3.1 Modelo RGB . . . 33
4.3.2 Notação hexadecimal . . . 33
4.3.3 Representação de cor em CSS . . . 33
4.3.4 Modelo CMYK . . . 34
4.3.5 Modelos HSV e HSL . . . 35
4.4 Profundidade de Cor e Canal Alfa . . . 35
4.4.1 Profundidade de cor (bit depth) . . . 35
4.4.2 Canal alfa e transparência . . . 36
4.5 Resolução e Densidade de Píxeis . . . 36
4.5.1 Resolução da imagem, PPI e DPI . . . 36
4.5.2 Cálculo de PPI — exemplos reais . . . 36
4.6 Factor Gama e Correcção de Gama . . . 37
4.6.1 Relação não-linear entre sinal e luminância . . . 37
4.6.2 Correcção de gama e o padrão sRGB . . . 37
4.7 Síntese do Capítulo . . . 38
4.8 Exercícios . . . 38

5 Layout CSS — Box Model, Flexbox e Grid 40
Introdução . . . 40
5.1 O Box Model . . . 40
5.1.1 Definição e camadas . . . 40
5.1.2 box-sizing: content-box vs border-box . . . 40
5.1.3 Margin, padding e border em CSS . . . 41
5.1.4 Colapso de margens . . . 42
5.2 Float e Grelha Manual . . . 42
5.2.1 O float e o seu propósito original . . . 42
5.2.2 O colapso do contentor e o clearfix . . . 43
5.2.3 Grelha de 12 colunas com floats . . . 43
5.2.4 Limitações do float como sistema de layout . . . 44
5.3 Flexbox . . . 44
5.3.1 O modelo flex . . . 44
5.3.2 Eixo principal e eixo transversal . . . 45
5.3.3 Propriedades do contentor flex . . . 45
5.3.4 Propriedades dos filhos flex . . . 46
5.3.5 Casos de uso típicos do Flexbox . . . 46
5.4 CSS Grid . . . 47
5.4.1 O modelo de grelha bidimensional . . . 47
5.4.2 Definir colunas e linhas . . . 47
5.4.3 Posicionamento de elementos . . . 48
5.4.4 Template areas . . . 49
5.4.5 Flexbox vs CSS Grid — quando usar cada um . . . 49
5.5 Síntese do Capítulo . . . 51

5.6 Exercícios 51

6 Frameworks CSS, Bootstrap e Micro-interações 53
Introdução 53
6.1 O Problema que os Frameworks Resolvem 53
6.1.1 CSS manual a escala 53
6.1.2 O que é um framework CSS 54
6.1.3 Panorama dos frameworks principais 54
6.2 Bootstrap — Instalação via CDN 54
6.2.1 O que é um CDN 54
6.2.2 Estrutura mínima de uma página Bootstrap 55
6.2.3 CSS personalizado sobre Bootstrap 55
6.3 O Sistema de Grid Bootstrap 56
6.3.1 Container, row e col 56
6.3.2 Os 6 breakpoints do Bootstrap 56
6.3.3 Classes responsivas — col-{bp}-{n} 57
6.4 Componentes Bootstrap 57
6.4.1 Navbar 57
6.4.2 Cards 58
6.4.3 Buttons 58
6.4.4 Forms 59
6.5 Utilitários Bootstrap 59
6.5.1 Spacing — margin e padding 59
6.5.2 Display, Flexbox e texto 60
6.6 Micro-interações em CSS Puro 60
6.6.1 O Bootstrap já anima — mas como? 60
6.6.2 transition 61
6.6.3 transform 62
6.6.4 Pseudo-classes como gatilhos 62
6.6.5 @keyframes — animações autónomas 63
6.6.6 Boas práticas e acessibilidade 64
6.7 Síntese do Capítulo 65
6.8 Exercícios 65

7 SASS Fundamental 67
Introdução 67
7.1 O Problema do CSS a Escala 67
7.1.1 Valores repetidos 67
7.1.2 Falta de estrutura 68
7.1.3 Manutenção difícil 68
7.2 O que é o SASS 68
7.2.1 Pré-processador CSS 68
7.2.2 Sintaxe SCSS 69
7.2.3 Compilação no fluxo de trabalho 69
7.3 Variáveis 69
7.3.1 Declaração e uso 69
7.3.2 Tipos de valores 70
7.3.3 Âmbito — global vs local 70
7.3.4 Variáveis SASS vs Custom Properties CSS 71
7.4 Nesting 72
7.4.1 Hierarquia no CSS 72
7.4.2 O `&` — referência ao selector pai 73

7.4.3 A regra dos 3 níveis . . . 73
7.5 Partials e Módulos . . . 74
7.5.1 Partials . . . 74
7.5.2 @import vs @use . . . 75
7.5.3 Módulos nativos do Dart Sass . . . 75
7.6 Mixins . . . 75
7.6.1 @mixin — bloco reutilizável . . . 75
7.6.2 Mixins com parâmetros . . . 76
7.6.3 Mixins com @content . . . 76
7.7 Extend e Placeholders . . . 77
7.7.1 @extend . . . 77
7.7.2 Placeholders % . . . 78
7.7.3 @extend vs @mixin — quando usar cada um . . . 78
7.8 Funções e Operadores . . . 79
7.8.1 Operadores aritméticos . . . 79
7.8.2 Funções de cor — color.adjust() . . . 79
7.8.3 Outras funções úteis . . . 80
7.9 Exemplo Integrado . . . 80
7.9.1 Estrutura do projecto . . . 80
7.9.2 _variables.scss — base de tudo {#sec-07-exemplo-variables} . . . 81
7.9.3 _components.scss — mixin e placeholder em acção {#sec-07-exemplo-components} . . . 81
7.9.4 _layout.scss — grelha responsiva {#sec-07-exemplo-layout} . . . 83
7.10 Síntese do Capítulo . . . 84
7.11 Exercícios . . . 84

Prefácio

Esta sebenta é um documento de estudo autónomo para a disciplina de **Ferramentas e Aplicações Multimédia**, cobrindo os principais temas de HTML/CSS, SASS, organização de projectos e desenvolvimento *web* moderno.

Organização da sebenta

A sebenta está organizada em quatro blocos temáticos:

1. **Fundamentos do Digital** — multimédia, codificação, sistemas numéricos, digitalização e compressão
2. **CSS e Arquitectura Visual** — layout, responsividade e folhas de estilo
3. **SASS e Organização Profissional** — pré-processamento CSS, estrutura de projecto, nomenclatura BEM
4. **Desenvolvimento SPA** — Angular, componentes, rotas, gestão de estado

Como usar esta sebenta

Os exemplos de código ao longo do texto são funcionais e podem ser executados directamente no *browser*. Os exercícios no final de cada capítulo destinam-se à consolidação autónoma dos conceitos apresentados.

Autores

João Pavão — UTAD

Sérgio Madeira — UTAD

1 Multimédia Digital — Fundamentos e Organização da Sebenta

Introdução

Esta sebenta acompanha a unidade curricular de **Ferramentas e Aplicações Multimédia** (FAM) do 1.º ano / 2.º semestre da Licenciatura em Comunicação e Multimédia da Universidade de Trás-os-Montes e Alto Douro. Os capítulos correspondem às aulas leccionadas e são disponibilizados progressivamente ao longo do semestre.

O presente capítulo serve dois propósitos: apresentar os fundamentos conceptuais da multimédia digital — base teórica necessária para todos os capítulos seguintes — e indicar a organização actual da sebenta, com uma descrição sumária dos conteúdos disponíveis.

1.1 O que é Multimédia?

1.1.1 Definição e integração de media

O termo *multimédia* é frequentemente mal interpretado como sinónimo de "muitos media" — a ideia de que basta combinar texto, imagem e som para ter multimédia. Esta visão é imprecisa. O que distingue a multimédia de uma simples colecção de ficheiros é a **integração coerente**: a combinação estruturada de diferentes formas de informação com um propósito comunicativo unificado.

Neste sentido, uma página web que integra texto editorial, fotografia, infografia animada e áudio narrativo é multimédia; a mesma página com os mesmos elementos dispostos sem relação entre si não o é. O critério determinante é a coerência da integração, não a contagem de tipos de media.

Os elementos que compõem um sistema multimédia digital incluem texto, imagem (estática e animada), som, vídeo e interactividade. A interactividade é o elemento que distingue o multimédia digital do multimédia analógico tradicional: permite que o utilizador participe na experiência, navegando, configurando e respondendo ao sistema.

1.1.2 Multimédia analógica vs digital

A transição do analógico para o digital não é apenas uma mudança de suporte — é uma mudança fundamental na natureza da representação da informação.

	Analógico	Digital
Sinal	Contínuo — varia suavemente	Discreto — valores separados

	Analógico	Digital
Exemplos	Jornal impresso, rádio, cinema	Website, podcast, streaming
Representação	Física directa (tinta, sulco, campo magnético)	Codificação numérica
Cópia	Degradação progressiva	Sem perda (bit a bit)
Transmissão	Degradação com distância	Robusta (detecção de erros)

O elemento central do digital é precisamente este: **tudo é representação codificada**. Um ficheiro `.jpg` não contém uma fotografia — contém uma sequência de números que, interpretada segundo a especificação JPEG, produz uma imagem quando renderizada. Um ficheiro `.mp3` não contém som — contém amostras de amplitude codificadas segundo o algoritmo MPEG Audio Layer III.

Esta característica tem uma consequência prática fundamental para a web: compreender o que está dentro de um ficheiro é condição necessária para escolher o formato certo, optimizar o desempenho e controlar a qualidade.

1.2 Tudo é Representação Numérica

1.2.1 O princípio fundamental

Dentro de um computador, nada existe na sua forma "real". Tudo — texto, imagem, som, vídeo — é armazenado e processado como sequências de números. Este princípio é a pedra angular da literacia digital:

- **Texto** → sequências de códigos numéricos (ASCII, UTF-8)
- **Imagem** → matriz de píxeis, cada um com um valor numérico de cor
- **Som** → sequência de amostras de amplitude ao longo do tempo
- **Vídeo** → sequência de imagens (píxeis) sincronizada com som

A consequência directa é que todo o processamento multimédia é, fundamentalmente, processamento matemático: filtrar uma imagem é aplicar operações matriciais; comprimir áudio é aplicar transformadas de frequência; transmitir vídeo é gerir fluxos de bits num canal com largura de banda finita.

1.2.2 Sinal analógico vs sinal digital

O mundo físico é analógico: os fenómenos que nos interessam — luz, som, temperatura — variam de forma contínua no tempo e na amplitude. Um microfone capta uma onda sonora como variação contínua de tensão eléctrica; uma câmara analógica regista luz como variação contínua de densidade química numa película.

Um computador, pelo contrário, só consegue armazenar valores discretos: um número finito de níveis, medidos em instantes separados. A conversão do contínuo para o discreto — a **digitalização** — implica sempre uma aproximação. Duas decisões determinam a qualidade dessa aproximação:

- **Com que frequência se mede** (frequência de amostragem): determina a resolução temporal do sinal digital.

- **Com que precisão se mede** (profundidade de bits): determina o número de níveis de amplitude disponíveis.

Mais amostras e mais bits produzem maior fidelidade — mas também mais dados. Surge assim o compromisso central do digital: **mais qualidade exige mais armazenamento e mais largura de banda**. A resposta a este compromisso é a compressão, tema desenvolvido em detalhe no capítulo seguinte.

1.2.3 O que é um ficheiro?

Um ficheiro é uma estrutura organizada de dados codificados segundo uma especificação, interpretável por software. Não há nada de intuitivamente "visual" ou "sonoro" num ficheiro — é apenas uma sequência de bytes.

O que torna um ficheiro `.jpg` diferente de um ficheiro `.mp3` não é o conteúdo bruto em bytes, mas a **especificação** que define como esses bytes devem ser interpretados. Um browser que abre um `.jpg` aplica o algoritmo de descompressão JPEG; um leitor de áudio que abre um `.mp3` aplica o descodificador MPEG. O mesmo ficheiro com a extensão errada permanece ilegível — a extensão é apenas uma indicação do formato, não uma propriedade intrínseca dos dados.

Esta distinção tem implicações práticas: a escolha do formato de ficheiro não é apenas uma questão de compatibilidade, mas de qualidade, tamanho e desempenho. Um ficheiro `.png` de uma fotografia e um ficheiro `.jpg` da mesma fotografia contêm a mesma imagem, mas com características radicalmente diferentes de tamanho, qualidade e capacidade de transparência.

1.3 A Web como Sistema Multimédia

1.3.1 Arquitectura da web

A web é o sistema multimédia mais abrangente da história. Sobre uma infraestrutura cliente-servidor, combina texto, imagem, som e vídeo numa experiência interactiva acessível a partir de qualquer dispositivo.

A construção de uma página web envolve quatro camadas complementares:

- **Infraestrutura** — servidor e cliente (browser): definem onde os dados residem e como são transferidos.
- **Estrutura** — HTML semântico: organiza o conteúdo em elementos com significado (parágrafo, cabeçalho, lista, tabela).
- **Apresentação** — CSS modular: define a aparência visual — tipografia, cor, layout, responsividade.
- **Conteúdo media** — imagem, som, vídeo: os elementos multimédia propriamente ditos, com os seus formatos, tamanhos e requisitos de desempenho específicos.

A interacção entre estas camadas é o objecto central desta UC.

1.3.2 A performance como parte do design

Um elemento frequentemente subestimado no design web é o desempenho. O tamanho dos ficheiros, a escolha de formatos e o grau de compressão têm impacto directo na velocidade de carregamento das páginas — e, por extensão, na experiência do utilizador e no posicionamento em motores de busca.

Num contexto onde um utilizador abandona uma página que demora mais de três segundos a carregar, a optimização de media deixa de ser um pormenor técnico e torna-se uma decisão de design. O digital é sempre um compromisso entre qualidade e velocidade; a literacia para navegar esse compromisso é uma das competências nucleares desta UC.

1.4 Estrutura da UC e da Sebenta

1.4.1 Programa e blocos temáticos

A UC organiza-se em quatro blocos temáticos sequenciais:

Bloco	Tema	Conteúdo central
1	Fundamentos do Digital	Multimédia, codificação, sistemas numéricos, digitalização, compressão, formatos
2	CSS e Arquitectura Visual	Layout, responsividade, metodologia BEM
3	SASS e Organização Profissional	Pré-processamento CSS, estrutura de projecto
4	Desenvolvimento SPA	Angular — componentes, rotas, gestão de estado

1.4.2 Avaliação

A classificação final (CF) calcula-se pela fórmula:

$$\mathrm{CF} = \mathrm{TT} \times 0{,}4 + \mathrm{TP1} \times 0{,}3 + \mathrm{TP2} \times 0{,}3$$

onde TT é o Teste Teórico (via Moodle, 40%), TP1 é o Trabalho Prático 1 (grupo de 3, 30%) e TP2 é o Trabalho Prático 2 (grupo de 3, 30%). A aprovação exige CF 9,5. Cada componente tem nota mínima obrigatória: TT 8,5; cada TP 7.

Existem três modos de avaliação:

- **Modo 1 — Avaliação Contínua:** condição TT 8,5, cada TP 7 e média prática 8,5; aprovação com CF 9,5.
- **Modo 2 — Avaliação Complementar:** para estudantes com prática 9,5 que repetem apenas o TT; aprovação com CF 9,5.
- **Modo 3 — Exame:** condição prática 8,5; aprovação com CF 9,5.

Em qualquer dos modos, o docente pode exigir defesa oral dos trabalhos a qualquer estudante, independentemente da nota obtida. Consultar o regulamento completo no Infoestudante.

1.5 Conteúdos Disponíveis na Sebenta

Os capítulos da sebenta são disponibilizados progressivamente ao longo do semestre, à medida que as aulas são leccionadas. A tabela seguinte lista os capítulos actualmente disponíveis e o seu conteúdo central.

> **Estado actual da sebenta**
>
> Esta sebenta encontra-se em construção activa. Os capítulos listados abaixo estão disponíveis para consulta. Novos capítulos serão adicionados ao longo do semestre.

Capítulo 2 — Sistemas Numéricos e Codificação

Apresenta os fundamentos matemáticos da representação digital: o sistema binário (base 2), o sistema hexadecimal (base 16) e as regras de conversão entre bases. Analisa a capacidade de representação em função do número de bits e aplica estes conceitos à codificação de texto (ASCII, UTF-8) e de cor (modelo RGB). Inclui demonstração interactiva de conversão entre bases e calculadora de capacidade de armazenamento.

Conceitos principais: bit, byte, binário, hexadecimal, conversão entre bases, ASCII, UTF-8, True Color, representação numérica de media.

Capítulo 3 — Digitalização e Compressão de Som e Imagem

Explica como os sinais analógicos contínuos são convertidos em sequências numéricas discretas (digitalização), analisando as três etapas do processo: amostragem, quantização e codificação. Apresenta o teorema de Nyquist e as consequências da sua violação (aliasing). Aborda a compressão sem perdas (Huffman, RLE) e com perdas (JPEG, MP3), com critérios práticos de escolha de formato para web e para arquivo.

Conceitos principais: amostragem, frequência de amostragem, teorema de Nyquist, quantização, profundidade de bits, compressão sem perdas, compressão com perdas, JPEG, PNG, SVG, WebP, WAV, MP3, AAC.

Capítulo 4 — Imagem Digital e Cor para Web

Aprofunda a representação da imagem digital e da cor em contexto web. Analisa a estrutura da imagem rasterizada (grelha de píxeis, fórmula de tamanho bitmap), compara imagens rasterizadas com imagens vectoriais (SVG), e detalha os quatro formatos bitmap principais para web (GIF, JPEG, PNG, WebP). Explica os fundamentos da percepção humana da cor (cones S/M/L, tricromacia), os modelos de cor RGB, CMYK, HSV e HSL, e a representação de cor em CSS. Aborda profundidade de cor, canal alfa, resolução, PPI/DPI e correcção de gama. Inclui cinco ferramentas interactivas (calculadora bitmap, conversor hexadecimal, demonstração CSS, selector HSL, calculadora PPI).

Conceitos principais: píxel, resolução, bpp, SVG, GIF, JPEG, PNG, WebP, cones S/M/L, síntese aditiva, RGB, CMYK, HSL, HSV, canal alfa, True Color, PPI, DPI, factor gama, sRGB.

1.6 Síntese do Capítulo

A multimédia digital não é uma colecção de ficheiros — é a integração coerente de diferentes formas de informação num sistema com propósito comunicativo. Toda a informação digital é representação numérica: texto, imagem, som e vídeo são, em última análise, sequências de bits codificadas segundo especificações.

A digitalização converte sinais contínuos em representações discretas, introduzindo um compromisso fundamental entre qualidade e volume de dados. A compressão é a resposta técnica a esse compromisso. A web é o sistema multimédia mais abrangente da actualidade, e a literacia para construir experiências digitais eficientes — escolhendo formatos, controlando tamanhos, optimizando desempenho — é o objectivo central desta UC.

Os capítulos seguintes desenvolvem cada um destes temas com rigor e com aplicação directa ao desenvolvimento web.

2 Sistemas Numéricos e Codificação

2.1 Motivação

No capítulo anterior estabeleceu-se uma ideia fundamental: tudo o que existe dentro de um computador é representação. Texto, imagem, som e vídeo não são armazenados na sua forma original — são convertidos em sequências de números e guardados como tal. Esta afirmação, porém, levanta uma questão imediata: que números são esses, e como funcionam?

Para compreender multimédia digital com rigor, não basta saber que "tudo são números". É necessário compreender *como* esses números são organizados, *que sistemas* os computadores utilizam para os representar, e *porquê* esses sistemas foram escolhidos. É precisamente esse o objectivo deste capítulo.

A abordagem parte do sistema decimal — o mais familiar — e progride para os sistemas binário e hexadecimal, pilares da representação digital. No final, analisa-se a forma como estes sistemas se aplicam directamente a texto, cores e outros elementos multimédia.

2.2 O Sistema Decimal

2.2.1 Representação posicional

O sistema decimal é o sistema numérico do quotidiano. É utilizado de forma tão natural que raramente se pensa na sua estrutura interna. No entanto, compreender essa estrutura é o ponto de partida para entender qualquer outro sistema numérico.

O sistema decimal é um sistema de **base 10**: utiliza dez dígitos distintos — 0, 1, 2, 3, 4, 5, 6, 7, 8 e 9 — e o valor de cada dígito depende da posição que ocupa no número. A este princípio chama-se **representação posicional**: cada posição corresponde a uma potência de 10, e o valor total do número é a soma dos produtos de cada dígito pela respectiva potência.

Considere-se o número 345:

$$345 = 3 \times 10^2 + 4 \times 10^1 + 5 \times 10^0 = 300 + 40 + 5$$

O dígito 3 não vale simplesmente "três" — vale trezentos, porque ocupa a posição das centenas (10^2). O dígito 4 vale quarenta, porque ocupa a posição das dezenas (10^1). O dígito 5 vale cinco, porque ocupa a posição das unidades (10^0).

Um segundo exemplo ilustra como esta decomposição funciona mesmo quando um dos dígitos é zero. No número 502:

$$502 = 5 \times 10^2 + 0 \times 10^1 + 2 \times 10^0 = 500 + 0 + 2$$

O zero na posição das dezenas não contribui para o valor total, mas é indispensável para preservar a posição correcta dos restantes dígitos. Esta ideia — a de que a posição determina o valor — será o fio condutor de todos os sistemas numéricos que se seguem.

2.3 O Sistema Binário

2.3.1 Por que base 2?

Os circuitos electrónicos que constituem um computador são, na sua essência, comutadores: dispositivos que se encontram num de dois estados físicos possíveis — conduzem corrente eléctrica ou não conduzem. A estes dois estados associam-se convencionalmente os valores **1** (ligado, tensão alta) e **0** (desligado, tensão baixa).

Esta limitação física não é uma desvantagem — é, pelo contrário, uma vantagem de engenharia: circuitos com apenas dois estados são mais simples, mais fiáveis e mais rápidos do que circuitos que tentassem distinguir dez níveis de tensão diferentes. A consequência directa é que os computadores representam toda a informação num sistema de **base 2**, designado **sistema binário**.

2.3.2 Estrutura do sistema binário

O sistema binário utiliza apenas dois dígitos: **0** e **1**. Cada um destes dígitos designa-se **bit**, abreviatura de *binary digit* — dígito binário. O bit é a unidade mínima de informação digital.

Tal como no sistema decimal, o valor de cada bit depende da posição que ocupa, sendo cada posição uma potência de 2. A capacidade de representação cresce exponencialmente com o número de bits: com n bits é possível representar 2^n valores distintos.

2.3.3 Conversão binário → decimal

A conversão de um número binário para decimal segue o mesmo princípio de expansão posicional apresentado para o sistema decimal, substituindo as potências de 10 por potências de 2.

Considere-se o número binário 1011_2:

Bit	Posição	Potência	Valor
1	3	2^3	8
0	2	2^2	0
1	1	2^1	2
1	0	2^0	1

$$1011_2 = 8 + 0 + 2 + 1 = 11_{10}$$

2.3.4 Conversão decimal → binário

A conversão inversa realiza-se através de **divisões sucessivas por 2**:

1. Dividir o número por 2 e registar o resto (0 ou 1).
2. Repetir a divisão com o quociente obtido.
3. Continuar até o quociente ser zero.
4. Ler os restos **de baixo para cima** — esse é o resultado binário.

Ilustra-se com a conversão de 219_{10}:

Divisão	Quociente	Resto
$219 \div 2$	109	**1**
$109 \div 2$	54	**1**
$54 \div 2$	27	**0**
$27 \div 2$	13	**1**
$13 \div 2$	6	**1**
$6 \div 2$	3	**0**
$3 \div 2$	1	**1**
$1 \div 2$	0	**1**

Lendo os restos de baixo para cima: $219_{10} = 11011011_2$.

Verificação: $11011011_2 = 128 + 64 + 0 + 16 + 8 + 0 + 2 + 1 = 219_{10}$✓

2.4 O Sistema Hexadecimal

2.4.1 Motivação e estrutura

À medida que os valores numéricos crescem, as suas representações binárias tornam-se longas e difíceis de ler. O número 219, que em decimal ocupa três dígitos, necessita de oito bits em binário. Endereços de memória, identificadores de cor e outros valores comuns em programação podem facilmente atingir 32 ou 64 bits — sequências praticamente ilegíveis para um ser humano.

O **sistema hexadecimal** surge como solução de legibilidade: um sistema de **base 16** com dezasseis símbolos — os dígitos 0 a 9 e as letras A a F:

Símbolo	A	B	C	D	E	F
Valor	10	11	12	13	14	15

O hexadecimal é amplamente utilizado em programação, na representação de endereços de memória, cores digitais em CSS e protocolos de rede.

2.4.2 Relação directa com o binário

$16 = 2^4$, o que significa que **4 bits correspondem exactamente a 1 dígito hexadecimal** — sem resíduos:

Binário	Hex	Binário	Hex
0000	0	1000	8
0001	1	1001	9
0010	2	1010	A
0011	3	1011	B
0100	4	1100	C
0101	5	1101	D
0110	6	1110	E
0111	7	1111	F

Esta tabela é tudo o que é necessário memorizar — substituição directa, sem qualquer cálculo.

2.4.3 Conversão decimal → hexadecimal (método directo)

Divisões sucessivas por 16, registando os restos de baixo para cima. Convertendo 219_{10}:

Divisão	Quociente	Resto
$219 \div 16$	13	**11 → B**
$13 \div 16$	0	**13 → D**

$219_{10} = \mathrm{DB}_{16}$ — mas os restos excedem 9 e têm de ser convertidos mentalmente para letra (11→B, 13→D). Em valores maiores, esta conversão acumula-se e torna-se propensa a erro.

2.4.4 Conversão decimal → hexadecimal (método prático)

$$\text{DECIMAL} \xrightarrow{\div 2} \text{BINÁRIO} \xrightarrow{4\,\text{bits}} \text{HEXADECIMAL}$$

Passo 1 — Converter para binário (restos sempre 0 ou 1, sem ambiguidade): $219_{10} = 11011011_2$

Passo 2 — Agrupar em blocos de 4 bits da direita para a esquerda:

$$\underbrace{1101}_{D}\,\underbrace{1011}_{B} \;\Rightarrow\; \mathrm{DB}_{16}$$

O resultado é idêntico, mas o processo é mais seguro. O binário actua como **ponte natural** entre o decimal e o hexadecimal.

2.4.5 Conversão hexadecimal → decimal (método directo)

Expansão posicional em potências de 16. Convertendo DB_{16}:

$$\mathrm{DB}_{16} = 13 \times 16^1 + 11 \times 16^0 = 208 + 11 = 219_{10}$$

O método funciona, mas exige cálculo de potências de 16 e conversão mental das letras. Para valores com três ou mais dígitos, torna-se progressivamente mais exigente.

2.4.6 Conversão hexadecimal → decimal (método prático)

$$\text{HEXADECIMAL} \xrightarrow{\text{4 bits}} \text{BINÁRIO} \xrightarrow{\text{expansão}} \text{DECIMAL}$$

Passo 1 — Substituição directa de cada dígito hex por 4 bits: $D \to 1101$, $B \to 1011 \;\Rightarrow$ $\mathrm{DB}_{16} = 11011011_2$

Passo 2 — Expansão posicional: $11011011_2 = 128 + 64 + 16 + 8 + 2 + 1 = 219_{10}$✓

2.4.7 Conversão binário → hexadecimal

A conversão mais simples — sem qualquer cálculo. Agrupam-se os bits em blocos de 4 da direita para a esquerda e substitui-se cada bloco:

$$\underbrace{1100}_{C}\underbrace{1101}_{D} \;\Rightarrow\; \mathrm{CD}_{16}$$

Se o número de bits não for múltiplo de 4, completa-se o bloco mais à esquerda com zeros sem alterar o valor.

2.4.8 O binário como ponte entre bases

O binário é a linguagem nativa dos computadores, o hexadecimal é a notação compacta para humanos, e o decimal é o sistema do mundo real. O binário é a ponte natural entre os três.

Conversão	Método directo	Via binário
DEC → BIN	Divisões por 2	—
BIN → DEC	Expansão posicional	—
DEC → HEX	Divisões por 16 (restos 0–F)	DEC → BIN → HEX
HEX → DEC	Expansão em potências de 16	HEX → BIN → DEC
BIN → HEX	—	Blocos de 4 bits
HEX → BIN	—	Substituição directa

Sempre que a conversão directa envolver base 16, o caminho via binário é mais simples e mais fiável.

2.5 Bits e Bytes

2.5.1 Definições

O **bit** é a unidade mínima de informação digital — um único dígito binário, com valor 0 ou 1. O agrupamento fundamental é o **byte**: um conjunto de 8 bits, unidade básica de armazenamento.

$$1\,\text{byte} = 8\,\text{bits} \qquad 2^8 = 256\,\text{valores possíveis}$$

2.5.2 Capacidade de representação

Bits	Valores possíveis	Aplicação típica
8	256	1 canal de cor RGB; 1 byte de texto
16	65 536	Áudio de alta fidelidade
24	16 777 216	Cor verdadeira — 3 canais RGB × 8 bits
32	4 294 967 296	Endereços IPv4; cor com canal alfa

2.6 Aplicação em Multimédia

2.6.1 Representação de texto — ASCII e Unicode

A representação de texto em computadores baseia-se numa ideia simples: associar um número inteiro a cada carácter. O sistema **ASCII** (*American Standard Code for Information Interchange*), desenvolvido na década de 1960, foi o primeiro padrão amplamente adoptado. Define 128 caracteres codificados em 7 bits.

Carácter	Decimal	Hexadecimal	Binário
A	65	41	01000001
B	66	42	01000010
a	97	61	01100001
0	48	30	00110000

Note-se que ‘A’ e ‘a’ têm códigos diferentes (65 e 97) — para o computador, são dois valores numéricos distintos.

O ASCII revelou-se insuficiente para representar todos os alfabetos do mundo. O **Unicode** define um espaço de codificação com mais de um milhão de pontos de código. A codificação **UTF-8**, a mais utilizada na web, é retrocompatível com o ASCII e representa os caracteres mais comuns em 1 byte, expandindo para 2, 3 ou 4 bytes conforme necessário.

2.6.2 Representação de cor — modelo RGB

O modelo **RGB** (vermelho, verde e azul — *Red, Green, Blue*) representa cada cor como combinação de três canais de 8 bits (valores 0–255). Um píxel RGB ocupa 24 bits (3 bytes), totalizando $2^{24} = 16\,777\,216$ cores — designadas **cor verdadeira** (*true color*).

Cor	R	G	B	Hexadecimal
Vermelho puro	255	0	0	#FF0000
Verde puro	0	255	0	#00FF00
Azul puro	0	0	255	#0000FF
Branco	255	255	255	#FFFFFF
Preto	0	0	0	#000000

A notação hexadecimal das cores é directamente legível: cada canal de 8 bits corresponde a exactamente 2 dígitos hexadecimais. `#FF0000` significa R=FF_{16}=255, G=00_{16}=0, B=00_{16}=0.

2.6.3 Representação de imagem, áudio e vídeo

Uma **imagem digital** é uma matriz bidimensional de píxeis, cada um representado por três bytes RGB. Uma imagem de 1920×1080 píxeis ocupa cerca de 6 MB sem compressão.

O **áudio digital** é uma sequência de amostras numéricas da amplitude de uma onda sonora — tipicamente de 16 bits no padrão de alta fidelidade, ou de 24 bits em produção profissional.

O **vídeo digital** combina uma sequência de imagens (fotogramas) com áudio sincronizado.

Em todos os casos: nada é "real" dentro do computador. Tudo são números — codificados e interpretados segundo convenções.

2.7 Síntese

- O **sistema binário** é a linguagem nativa dos computadores, motivada pela natureza física dos circuitos electrónicos.
- O **sistema hexadecimal** é uma notação compacta baseada na equivalência directa de 4 bits por dígito.
- O **binário funciona como ponte** entre decimal e hexadecimal: sempre que uma conversão directa envolver base 16, o caminho via binário é mais simples e mais fiável.
- Com n bits representam-se 2^n valores distintos.
- Toda a informação multimédia é, em última análise, uma sequência de números binários interpretados segundo convenções de codificação.

2.8 Exercícios

2.8.1 Conversões numéricas

Exercício 2.1 — Binário para decimal

Converta os seguintes números binários para decimal, mostrando a expansão posicional completa:

a) 10110100_2
b) 11111111_2
c) 10000001_2

Verificação: 180, 255 e 129.

Exercício 2.2 — Decimal para binário

Converta para binário pelo método das divisões sucessivas por 2:

a) 137_{10}
b) 200_{10}
c) 255_{10}

Verificação: 10001001_2, 11001000_2 e 11111111_2.

Exercício 2.3 — Hexadecimal

Realize as seguintes conversões, usando o binário como ponte sempre que adequado:

a) $A3_{16} \rightarrow$ decimal
b) $187_{10} \rightarrow$ hexadecimal
c) $FF_{16} \rightarrow$ decimal
d) $1F4_{16} \rightarrow$ decimal

Verificação: 163, BB_{16}, 255 e 500.

2.8.2 Aplicação em multimédia

Exercício 2.4 — Cores RGB

A cor de fundo de um elemento CSS é `#3A7FCC`.

a) Identifique os valores decimais dos três canais RGB.
b) Qual o canal dominante? O que isso implica visualmente?
c) Quantas cores distintas são representáveis com 24 bits?

Orientação para b): O canal B — $CC_{16} = 204$ — é o dominante. A cor tende para tons azulados.

Exercício 2.5 — Capacidade de representação

Uma imagem não comprimida tem 800×600 píxeis a 24 bits por píxel.

a) Quantos píxeis tem a imagem?
b) Quantos bytes ocupa sem compressão?
c) E em kilobytes? E em megabytes?

3 Digitalização e Compressão de Som e Imagem

3.1 Enquadramento

No capítulo anterior estabeleceu-se que toda a informação digital é representação numérica: texto, cor, imagem e áudio traduzem-se em sequências de bits organizados segundo convenções de codificação. Ficou, porém, por responder uma questão essencial: como é que os sinais do mundo real — contínuos, analógicos, ricos em detalhe — se convertem nessas sequências de números?

É precisamente esse processo — a **digitalização** — o objecto deste capítulo. Analisa-se também a consequência imediata da digitalização: os ficheiros resultantes são volumosos, e a sua transmissão e armazenamento exigem **compressão**. No final, apresentam-se os principais formatos de imagem e áudio utilizados na web, com critérios práticos de escolha.

3.2 Sinais no Mundo Real

3.2.1 Sinais contínuos e sinais discretos

A maior parte dos fenómenos naturais que interessam à multimédia — som, luz, temperatura, pressão — são **sinais contínuos**: variam em todos os instantes do tempo e podem assumir infinitos valores possíveis. Uma onda sonora, por exemplo, existe de forma ininterrupta na natureza, com amplitude que varia suavemente a cada fracção de segundo.

Os computadores, pelo contrário, trabalham com **sinais discretos**: representações compostas por valores separados, num número finito de níveis. Para armazenar, transmitir e processar um sinal natural num computador, é necessário convertê-lo de contínuo para discreto. Esse processo chama-se digitalização.

	Sinal contínuo	Sinal discreto
Variação	Contínua em todos os instantes	Valores separados
Amplitude	Infinitos valores possíveis	Número finito de níveis
Exemplo	Onda sonora analógica	Áudio digital amostrado

3.3 Digitalização

3.3.1 O processo de digitalização

Digitalizar significa converter um sinal contínuo num conjunto de valores numéricos que o representam com suficiente fidelidade. Este processo é o que permite armazenar música num ficheiro, gravar vídeo numa câmara ou transmitir voz por uma chamada digital.

A digitalização decompõe-se em três etapas sequenciais:

$$\text{Sinal contínuo} \xrightarrow{\text{Amostragem}} \xrightarrow{\text{Quantização}} \xrightarrow{\text{Codificação}} \text{Sequência de bits}$$

Cada etapa introduz uma aproximação ao sinal original — a qualidade final depende das escolhas feitas em cada uma delas.

3.4 Amostragem

3.4.1 O que é a amostragem

A **amostragem** consiste em medir o valor do sinal em intervalos regulares de tempo. Em vez de registar o sinal de forma contínua — o que seria impossível num sistema digital — recolhe-se uma *amostra* a cada intervalo fixo. Quanto mais amostras por segundo, mais fielmente o sinal original é representado.

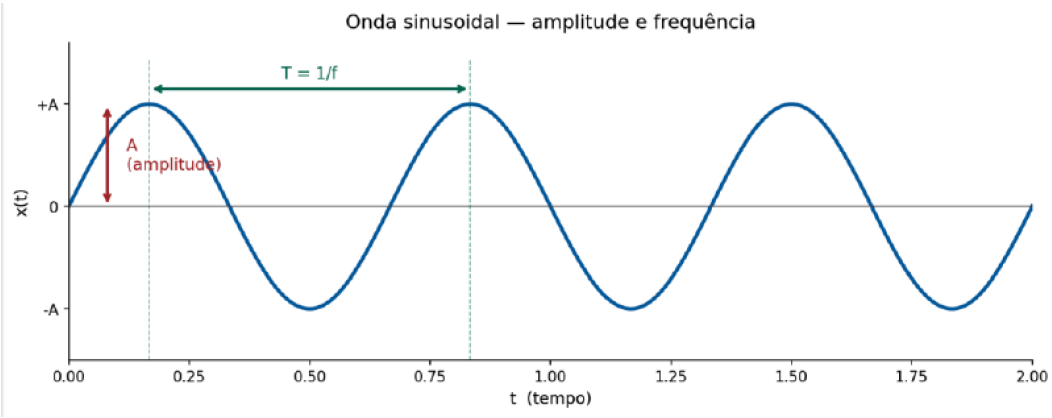


Figura 3.1: Exemplo de sinal contínuo. O tempo T que demora uma onda a repetir o seu estado anterior é o período da onda. T é o inverso da frequência

3.4.2 Frequência de amostragem

O número de amostras recolhidas por segundo designa-se **frequência de amostragem** e mede-se em hertz (Hz) — ou, mais correctamente, em amostras por segundo. A relação é directa: frequência mais alta significa maior fidelidade ao sinal original, mas também maior volume de dados.

Contexto	Frequência	Amostras/s
Telefone	8 kHz	8 000

Contexto	Frequência	Amostras/s
Áudio CD	44,1 kHz	44 100
Áudio HD	96–192 kHz	96 000–192 000

O padrão de 44,1 kHz do CD foi estabelecido com base no teorema de Nyquist, como se explica na secção seguinte.

3.4.3 Teorema de Nyquist

O teorema de Nyquist-Shannon é o fundamento teórico da amostragem digital. Enuncia-se da seguinte forma:

> Para representar correctamente um sinal, a frequência de amostragem deve ser pelo menos o dobro da frequência máxima presente no sinal.

$$f_a \geq 2 \times f_{\text{máx}}$$

A título de exemplo: a voz humana contém frequências até aproximadamente 4 kHz; para a digitalizar correctamente, é necessário amostrar a pelo menos 8 kHz — daí o padrão telefónico. O ouvido humano percebe frequências até cerca de 20 kHz; o padrão CD de 44,1 kHz cobre esse intervalo com margem de segurança.

Quando a frequência de amostragem é inferior ao limiar de Nyquist, ocorre **aliasing**: o sinal reconstruído tem uma frequência errada — diferentes das do sinal original — porque as amostras são insuficientes para distinguir os ciclos do sinal.

> **!** Importante
>
> O aliasing é uma distorção **irreversível** — uma vez que o sinal foi amostrado abaixo do limiar de Nyquist, a informação perdida não pode ser recuperada.

3.5 Quantização

3.5.1 O que é a quantização

Após a amostragem, cada valor medido é um número real — pode assumir qualquer valor dentro de um intervalo contínuo. O sistema digital, porém, só consegue armazenar valores inteiros num conjunto finito de níveis. A **quantização** é o processo de mapear cada valor amostrado para o nível discreto mais próximo dentro dessa escala finita.

Intuitivamente: imagine uma régua com marcações apenas nos centímetros inteiros. Qualquer medida (13,7 cm, 8,3 cm) tem de ser arredondada para o valor marcado mais próximo (14 cm, 8 cm). Quanto mais marcações a régua tiver, menor o erro de arredondamento.

O número de níveis disponíveis é determinado pela **profundidade de bits**: com n bits obtêm-se 2^n níveis distintos.

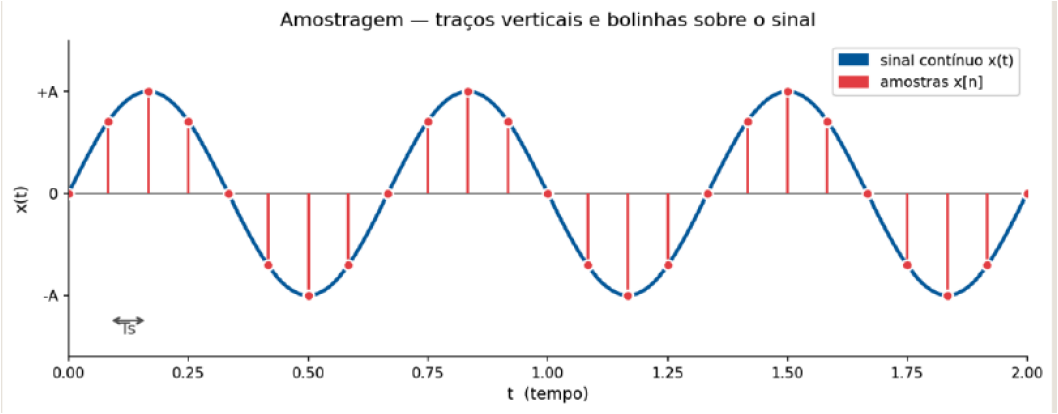


Figura 3.2: Amostras discretas do sinal contínuo da figura anterior. Cada traço vermelho vertical representa uma amostra recolhida num instante de tempo.

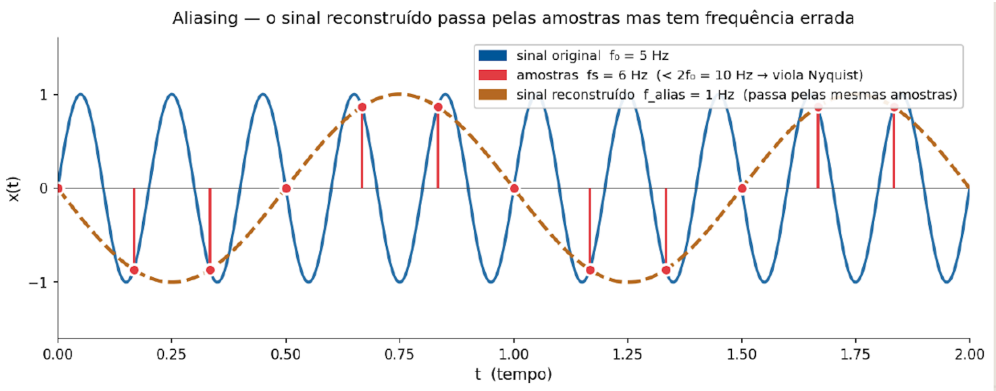


Figura 3.3: Aliasing: quando a frequência de amostragem viola o teorema de Nyquist, o sinal reconstruído (tracejado laranja) tem frequência diferente do sinal original (azul), apesar de passar pelas mesmas amostras (pontos vermelhos).

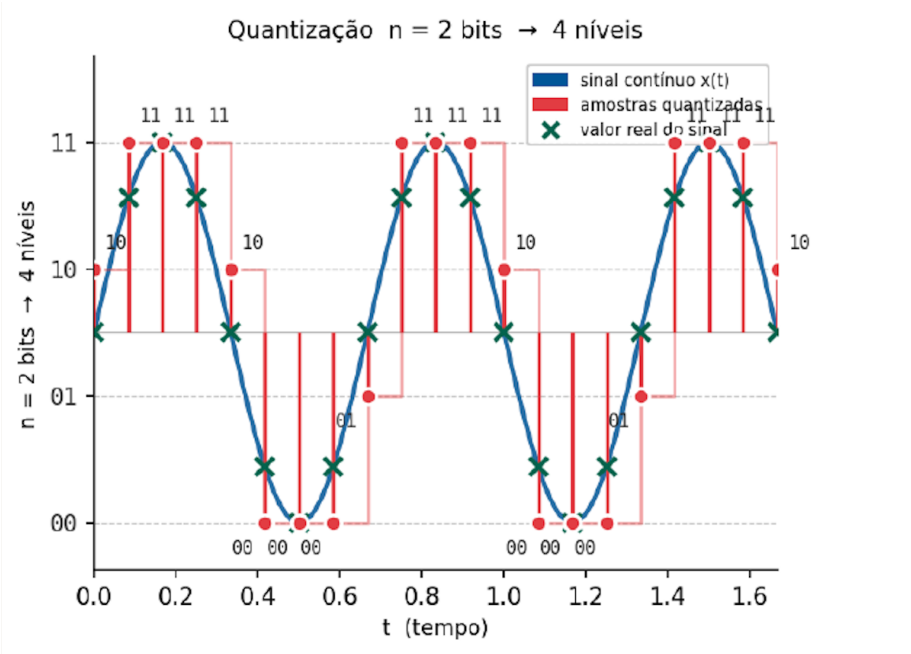


Figura 3.4: Quantização com 2 bits (4 níveis). A aproximação ao sinal contínuo original é muito rudimentar.

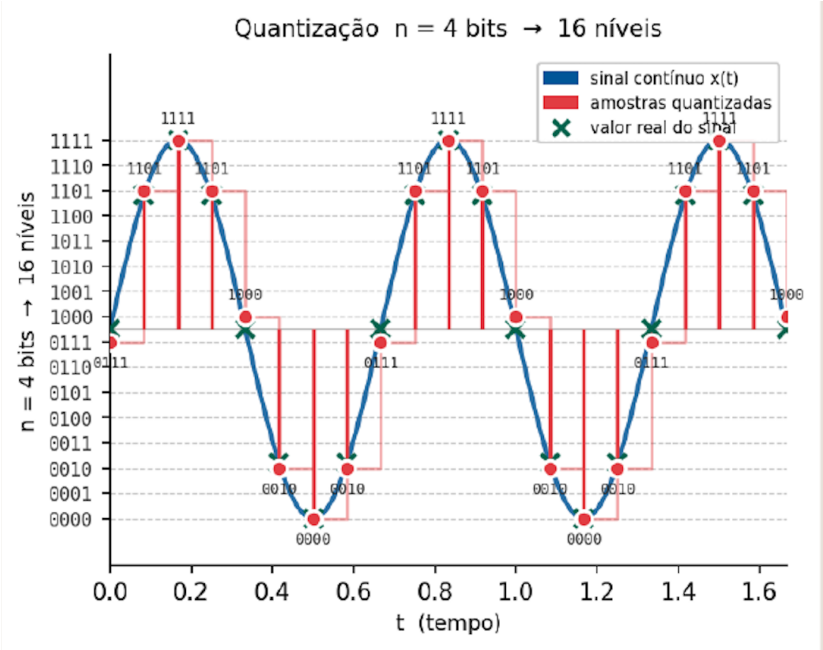


Figura 3.5: Comparação entre quantização com 4 bits (16 níveis). Com mais níveis, a aproximação ao sinal contínuo original é mais fiel.

3.5.2 Profundidade de bits

Profundidade	Níveis	Aplicação típica
8 bits	256	Telefone, rádio AM
16 bits	65 536	Padrão CD — qualidade elevada
24 bits	16 777 216	Estúdio profissional

O padrão CD combina amostragem a 44,1 kHz com profundidade de 16 bits — uma escolha que cobre a gama perceptível pelo ouvido humano com qualidade suficiente para a grande maioria dos contextos.

3.6 Compressão

3.6.1 A necessidade de compressão

Os ficheiros multimédia não comprimidos ocupam volumes de dados consideráveis. Para dimensionar o problema, calculam-se dois exemplos representativos:

Imagem 1920×1080 px a 24 bits:

Passo	Operação	Resultado
Píxeis totais	1920 × 1080	2 073 600 px
Bits totais	2 073 600 × 24	49 766 400 bits
Bytes	49 766 400 ÷ 8	6 220 800 bytes
Megabytes	6 220 800 ÷ 1 048 576	**5,93 MB**

Áudio WAV estéreo — 44,1 kHz / 16 bits / 1 minuto:

Passo	Operação	Resultado
Amostras por canal	44 100 × 60 s	2 646 000
Amostras totais	2 646 000 × 2 canais	5 292 000
Bits totais	5 292 000 × 16	84 672 000 bits
Megabytes	84 672 000 ÷ 8 ÷ 1 048 576	**10,09 MB**

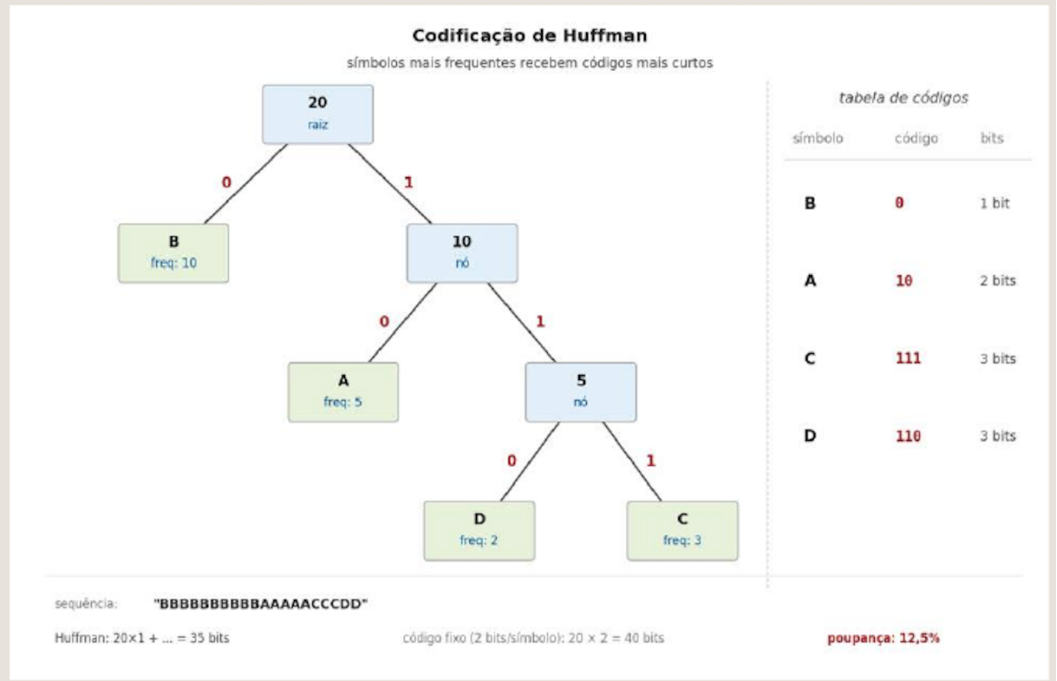
Um filme de duas horas em qualidade HD não comprimida ocuparia na ordem dos 200 GB — inviável para transmissão ou armazenamento doméstico. Daí a necessidade de compressão.

3.6.2 Compressão sem perdas

A **compressão sem perdas** (*lossless compression*) elimina apenas a redundância presente nos dados — padrões repetidos, sequências previsíveis, informação que pode ser reconstruída matematicamente. Os dados originais são recuperáveis na íntegra após descompressão.

Dois algoritmos clássicos ilustram os princípios fundamentais:

Codificação de Huffman — atribui códigos binários mais curtos aos símbolos mais frequentes e códigos mais longos aos mais raros. Para a sequência `BBBBBBBBBBAAAACCCDD`, o símbolo B (frequência 10) recebe o código `0` (1 bit), enquanto D (frequência 2) recebe `110` (3 bits) — poupança de 12,5% face ao código fixo.



RLE (Run-Length Encoding) — substitui sequências de símbolos repetidos pelo par (contagem, símbolo). A sequência de 14 caracteres `AAABBBBBCCDDDDD` é codificada como `3A5B2C4D` — 8 caracteres, poupança de 43%.

Outros algoritmos sem perdas amplamente utilizados incluem LZ77, LZ78 e DEFLATE (este último é a base dos formatos ZIP e PNG).

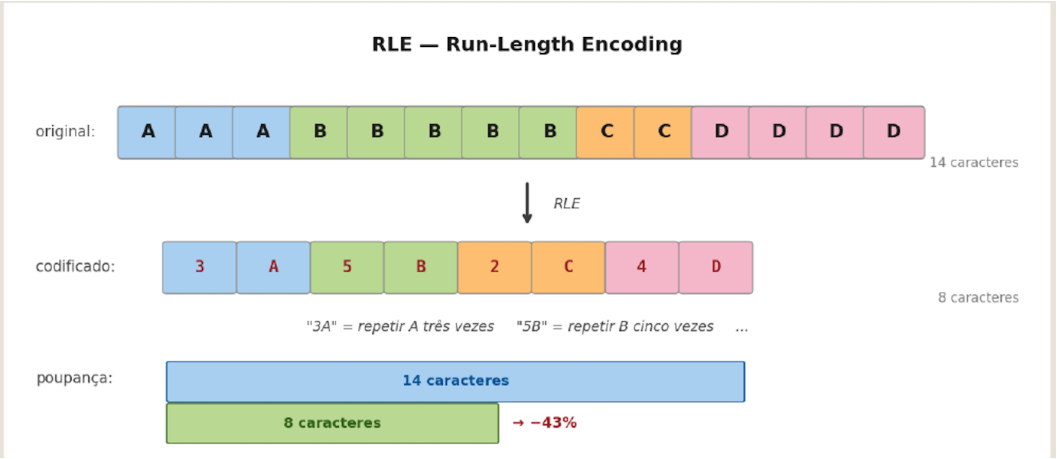


Figura 3.6: Codificação de Huffman (esquerda) e RLE — Run-Length Encoding (direita). Ambos são algoritmos de compressão sem perdas.

Quando usar compressão sem perdas: documentos de texto, gráficos com áreas de cor uniforme, imagens com texto, logótipos, e qualquer situação em que a qualidade máxima é obrigatória.

3.6.3 Compressão com perdas

A **compressão com perdas** (*lossy compression*) vai mais longe: remove informação que os modelos da percepção humana consideram pouco perceptível. O resultado é um ficheiro substancialmente mais pequeno, mas a qualidade original não é recuperável.

Os algoritmos de compressão com perdas exploram limitações do sistema perceptivo humano — o olho é menos sensível a variações de alta frequência em regiões de baixo contraste, o ouvido é menos sensível a sons fracos mascarados por sons fortes, etc. Formatos como JPEG (imagem), MP3 e AAC (áudio) e H.264 (vídeo) baseiam-se neste princípio.

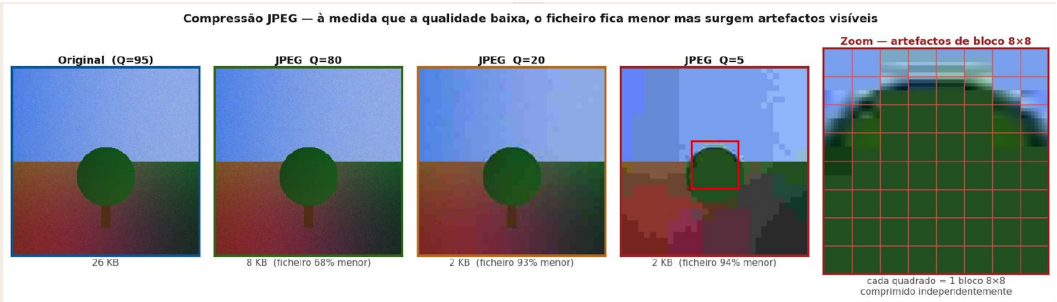


Figura 3.7: Efeito da compressão JPEG a diferentes níveis de qualidade. A Q=5, surgem artefactos de bloco 8×8 bem visíveis — cada bloco é comprimido independentemente.

Limitações da compressão com perdas:

- A qualidade original não é recuperável após compressão
- Artefactos tornam-se visíveis a taxas de compressão muito elevadas
- Compressões repetidas degradam progressivamente a qualidade (*generation loss*)

3.6.4 O compromisso qualidade vs. tamanho

Não existe uma escolha universalmente correcta entre qualidade e tamanho — depende do contexto de uso:

Contexto	Prioridade	Escolha típica
Arquivo	Qualidade máxima	Sem perdas (PNG, WAV)
Web	Equilíbrio	JPEG médio, WebP
Streaming	Tamanho mínimo	Compressão elevada
Edição	Sem degradação	Sem perdas ou RAW

3.7 Formatos de Imagem

A escolha do formato de imagem afecta directamente a qualidade visual, o tamanho do ficheiro e a compatibilidade com browsers e ferramentas. Apresentam-se os quatro formatos fundamentais para a web.

3.7.1 JPEG

O **JPEG** (*Joint Photographic Experts Group*) é o formato dominante para fotografia digital. Utiliza compressão com perdas baseada na transformada de cosseno discreta (DCT), que opera em blocos de 8×8 píxeis.

- Compressão com perdas, controlada por um parâmetro de qualidade
- Excelente para fotografias e imagens com gradientes suaves
- Suporte universal em todos os browsers
- Não suporta transparência (canal alfa)
- Não adequado para imagens com texto ou bordas nítidas

Usar quando: fotografias, imagens realistas, imagens com gradientes, quando o tamanho do ficheiro é prioritário.

3.7.2 PNG

O **PNG** (*Portable Network Graphics*) foi criado como alternativa sem perdas ao GIF. Utiliza o algoritmo DEFLATE para compressão.

- Compressão sem perdas — qualidade original preservada
- Suporta transparência total (canal alfa de 8 bits)
- Ideal para gráficos, interface de utilizador e imagens com texto
- Ficheiros tipicamente maiores que JPEG para fotografias

Usar quando: logótipos, ícones, capturas de ecrã, imagens com texto, elementos de interface, quando a transparência é necessária.

3.7.3 SVG

O **SVG** (*Scalable Vector Graphics*) é fundamentalmente diferente dos formatos anteriores: não armazena píxeis, mas sim descrições matemáticas de formas, curvas e texto em formato XML.

- Formato vectorial — escala sem qualquer perda de qualidade
- Editável directamente com código ou editor de texto
- Tamanho muito reduzido para gráficos simples
- Suporta animações via CSS e JavaScript
- Não adequado para fotografias

Usar quando: ícones, logótipos, ilustrações para ecrã, diagramas, animações, quando há múltiplas resoluções de ecrã.

3.7.4 WebP

O **WebP** foi desenvolvido pela Google em 2010 como formato moderno para a web, suportando tanto compressão com perdas como sem perdas.

- Com ou sem perdas, conforme a configuração
- 20–34% mais pequeno que JPEG na mesma qualidade perceptível
- Suporta transparência (como PNG) e animação (como GIF)
- Suporte nos browsers modernos (Chrome, Firefox, Safari, Edge)

Usar quando: web moderna, aplicações web progressivas (PWA), quando o desempenho de carregamento é crítico.

3.8 Som Digital

3.8.1 Representação do som digital

O som digital é uma sequência de amostras numéricas que representam a amplitude de uma onda sonora ao longo do tempo. Cada amostra é um valor inteiro que indica a pressão sonora naquele instante.

O tamanho de um ficheiro áudio não comprimido calcula-se por:

$$\text{Tamanho} = f_a \times n_{\text{bits}} \times n_{\text{canais}} \times t_{\text{segundos}}$$

onde f_a é a frequência de amostragem, n_{bits} a profundidade de bits, n_{canais} o número de canais (1 para mono, 2 para estéreo) e t a duração em segundos.

3.8.2 Formatos áudio

WAV — formato sem compressão (PCM puro). Preserva a qualidade máxima e é o padrão para edição profissional. Ficheiros grandes.

MP3 — compressão com perdas baseada em modelos psicoacústicos. Muito compacto e amplamente suportado. Perde frequências altas a taxas de compressão elevadas. Formato dominante durante décadas.

AAC (*Advanced Audio Coding*) — compressão com perdas de geração posterior ao MP3. Melhor qualidade à mesma taxa de transferência. Padrão adoptado pela Apple e pelo YouTube.

Formato	Compressão	Qualidade	Tamanho	Uso típico
WAV	Nenhuma	Máxima	Grande	Edição, arquivo
MP3	Com perdas	Boa	Pequeno	Distribuição
AAC	Com perdas	Muito boa	Pequeno	Streaming, móvel

3.9 Síntese

- A **digitalização** converte sinais analógicos contínuos em sequências numéricas através de três etapas: amostragem, quantização e codificação.
- O **teorema de Nyquist** estabelece que a frequência de amostragem deve ser pelo menos o dobro da frequência máxima do sinal — abaixo deste limiar ocorre aliasing irreversível.
- A **profundidade de bits** determina o número de níveis de quantização: com n bits obtêm-se 2^n níveis distintos.
- A **compressão sem perdas** elimina redundância preservando os dados originais na íntegra; a **compressão com perdas** remove informação pouco perceptível para obter ficheiros substancialmente mais pequenos.
- A escolha do **formato** correcto — JPEG, PNG, SVG, WebP, WAV, MP3, AAC — afecta qualidade, tamanho e desempenho, e depende do contexto de utilização.

3.10 Exercícios

3.10.1 Digitalização e amostragem

Exercício 3.1 — Teorema de Nyquist

Determine a frequência de amostragem mínima para cada caso:

a) Sinal de áudio com frequência máxima de 8 kHz
b) Sinal de vídeo com frequência máxima de 6 MHz
c) Sinal de voz humana com frequência máxima de 4 kHz

Verificação: 16 kHz, 12 MHz e 8 kHz.

Exercício 3.2 — Tamanho de ficheiros áudio

Calcule o tamanho em megabytes de um ficheiro WAV estéreo com as seguintes características:

a) 44,1 kHz / 16 bits / 3 minutos
b) 96 kHz / 24 bits / 1 minuto

Verificação: 30,3 MB e 34,3 MB.

3.10.2 Compressão e formatos

Exercício 3.3 — Escolha de formato

Para cada situação, indique o formato de imagem mais adequado e justifique a escolha:

a) Fotografia de produto para uma loja online
b) Logótipo de empresa para o cabeçalho de um site
c) Ícone de interface que deve funcionar em ecrãs de alta densidade
d) Captura de ecrã de documentação técnica com texto

Exercício 3.4 — Compressão sem perdas

Aplique o algoritmo RLE à seguinte sequência de 20 caracteres:
`AAAAABBBBBBBCCDDDDDD`

a) Qual é a representação RLE resultante?
b) Quantos caracteres ocupa a representação original?
c) Quantos caracteres ocupa a representação RLE?
d) Qual a taxa de compressão obtida?

Verificação: `5A7B2C6D` — 8 caracteres; taxa de compressão de 60%.

4 Imagem Digital e Cor para Web

Introdução

A representação digital da cor e da imagem é um dos pilares fundamentais do design para a web. Compreender como uma fotografia se converte numa grelha de valores numéricos, como o olho humano percebe a cor, e como os diferentes modelos cromáticos se traduzem em código CSS constitui um saber indispensável para qualquer profissional de multimédia.

Neste capítulo analisa-se a estrutura de uma imagem digital, os principais formatos de ficheiro para uso em contexto web, os fundamentos da percepção humana da cor, os modelos de cor utilizados em sistemas digitais e na impressão, e os conceitos de profundidade cromática, resolução e correcção de gama.

4.1 A Imagem Digital

4.1.1 Definição e estrutura de uma imagem rasterizada

Uma imagem digital rasterizada é uma grelha rectangular de elementos discretos — os **píxeis** (*picture elements*) — cada um dos quais armazena um valor numérico de cor. A organização em linhas e colunas permite que qualquer imagem seja representada como uma matriz bidimensional de números inteiros, processável computacionalmente.

A **resolução** de uma imagem exprime-se pelo produto largura × altura em píxeis. Uma imagem Full HD com resolução 1920 × 1080 contém 1 920 × 1 080 = 2 073 600 píxeis, o equivalente a cerca de dois megapíxeis. Quanto maior o número de píxeis, maior o detalhe que pode ser representado — mas também maior o volume de dados gerado.

> **Terminologia**
>
> O termo **píxel** deriva da contracção de *picture element*. É a unidade mínima e indivisível de uma imagem rasterizada. Cada píxel possui um único valor de cor, definido pelo modelo cromático em uso.

4.1.2 Tamanho de um ficheiro bitmap

O tamanho teórico de uma imagem bitmap não comprimida calcula-se pela fórmula:

$$T = \text{largura} \times \text{altura} \times \text{bpp}$$

onde bpp (*bits per pixel*) é a profundidade de cor. O resultado obtém-se em bits; para converter para bytes divide-se por 8; para kilobytes divide-se novamente por 1 024; para megabytes divide-se mais uma vez por 1 024.

Exemplo 1 — imagem 640 × 480 a 24 bpp (True Color):

$$T = 640 \times 480 \times 24 = 7\,372\,800 \text{ bits} = 921\,600 \text{ bytes} \approx 900 \text{ KB}$$

Exemplo 2 — imagem Full HD 1920 × 1080 a 24 bpp:

$$T = 1920 \times 1080 \times 24 = 49\,766\,400 \text{ bits} = 6\,220\,800 \text{ bytes} \approx 5{,}93 \text{ MB}$$

O segundo exemplo evidencia por que motivo a compressão é indispensável: uma única imagem Full HD sem compressão ocupa quase 6 MB. Na web, onde uma página pode conter dezenas de imagens, o impacto no tempo de carregamento seria inaceitável.

A calculadora seguinte permite verificar estes cálculos de forma interactiva:

4.1.3 Imagem rasterizada vs imagem vetorial

Existem dois paradigmas fundamentais de representação de imagem digital: o **rasterizado** (ou bitmap) e o **vetorial**.

Numa imagem rasterizada, a informação está armazenada píxel a píxel. Esta abordagem é adequada para fotografias e imagens com gradientes complexos, mas apresenta uma limitação estrutural: ao ampliar a imagem além da sua resolução nativa, o sistema não dispõe de informação adicional e os píxeis tornam-se visíveis — o fenómeno vulgarmente designado por *pixelização*. Da mesma forma, a perda de píxeis por corrupção de dados é irreversível.

Numa imagem vetorial, a informação é armazenada sob a forma de descrições matemáticas: coordenadas de pontos, equações de curvas, parâmetros de formas geométricas. Estas descrições são independentes da resolução de saída — a imagem pode ser ampliada ou reduzida sem qualquer perda de qualidade, porque o sistema recalcula os píxeis a cada renderização. O tamanho do ficheiro é determinado pela complexidade da descrição matemática, não pelas dimensões da imagem.

Característica	Rasterizado (Bitmap)	Vetorial
Unidade base	Píxel	Fórmula matemática
Ampliação	Perde qualidade	Sem perda
Fotografia	Ideal	Impróprio
Logótipos e ícones	Limitado	Ideal
Formatos típicos	JPEG, PNG, WebP, GIF	SVG
Tamanho do ficheiro	Proporcional à resolução	Proporcional à complexidade
Corrupção parcial	Píxeis afectados	Pode destruir o ficheiro

4.1.4 Formatos de imagem bitmap para web

Os quatro formatos bitmap dominantes na web distinguem-se pelas características de compressão, suporte de transparência e casos de uso adequados.

GIF (*Graphics Interchange Format*) utiliza uma profundidade de 8 bpp, limitando a paleta a 256 cores indexadas. A compressão é sem perdas (algoritmo LZW). Suporta animação e transparência binária (um píxel é totalmente transparente ou totalmente opaco). Adequado para animações simples e ícones com poucas cores; inadequado para fotografias.

JPEG (*Joint Photographic Experts Group*) suporta até 24 bpp com compressão com perdas ajustável por parâmetro de qualidade. Não suporta transparência nem animação. É o formato de eleição para fotografias e imagens com gradientes suaves, sendo inadequado para zonas de cor uniforme ou imagens com texto, onde os artefactos de compressão são perceptíveis.

PNG (*Portable Network Graphics*) trabalha geralmente a 32 bits, com três canais RGB de 8 bits mais um canal alfa de 8 bits. A compressão é sem perdas; o formato não é proprietário. Adequado para gráficos de interface, imagens com texto e elementos que requerem transparência.

WebP foi desenvolvido pela Google em 2010 como substituto moderno. Suporta compressão com e sem perdas, canal alfa e animação, produzindo ficheiros tipicamente 20 a 34% mais pequenos que JPEG na mesma qualidade perceptível. É o formato recomendado para novas aplicações web.

4.1.5 Imagens vetoriais — SVG

O **SVG** (*Scalable Vector Graphics*) é o formato vetorial padrão para a web. Baseia-se em XML, o que significa que o conteúdo é texto simples, editável directamente, incorporável em HTML, e manipulável via CSS e JavaScript.

As principais vantagens do SVG incluem a escalabilidade sem degradação (fundamental para ecrãs Retina), o tamanho reduzido para formas simples, e a integração nativa no DOM. As desvantagens são a inadequação para fotografias e o crescimento do ficheiro com a complexidade da cena — ao contrário dos formatos bitmap, o tamanho de um SVG não depende das dimensões da imagem, mas do número de formas que a compõem.

A estrutura básica de um ficheiro SVG é a seguinte:

```svg
<!-- SVG mínimo: círculo azul com rectângulo escuro -->
<svg width="200" height="120" xmlns="http://www.w3.org/2000/svg">

  <!-- Círculo: centro (cx, cy), raio r, cor de preenchimento fill -->
  <circle cx="100" cy="55" r="45" fill="#3366CC"/>

  <!-- Rectângulo: posição (x, y), dimensões, cor -->
  <rect x="20" y="80" width="160" height="25" fill="#2C3E50"/>

  <!-- Texto centrado -->
  <text x="100" y="97" text-anchor="middle"
        fill="white" font-size="12" font-family="sans-serif">SVG</text>

</svg>
```

Os usos típicos de SVG em contexto web são logótipos, ícones, ilustrações, gráficos de dados interactivos, animações leves e interfaces para ecrãs de alta densidade.

Critérios de escolha de formato:

- Fotografia, imagem real → JPEG ou WebP
- Gráfico com transparência → PNG
- Logótipo, ícone, diagrama → SVG
- Animação simples → GIF ou WebP animado

4.1.6 Entrelaçamento (Progressive Rendering)

O **entrelaçamento** (*interlacing* ou *progressive rendering*) é uma técnica que permite apresentar uma imagem gradualmente durante o carregamento: em vez de a imagem surgir linha por linha de cima para baixo (comportamento sequencial *baseline*), surge primeiro uma versão de baixa resolução que vai ganhando detalhe à medida que chegam mais dados. Esta abordagem melhora a experiência percepcionada do utilizador em ligações lentas, pois torna imediatamente visível uma versão reconhecível da imagem.

Os três principais formatos bitmap para web implementam o entrelaçamento de forma distinta:

- **GIF** utiliza o método Adam7 em 4 passagens por linhas: 1.ª passagem (linhas 0, 8, 16, …), 2.ª passagem (linhas 4, 12, 20, …), 3.ª passagem (linhas 2, 6, 10, …) e 4.ª passagem (linhas 1, 3, 5, …).
- **JPEG progressivo** codifica a imagem em múltiplas varriduras de frequência crescente: a primeira varrida transmite uma versão global de baixa qualidade e as seguintes acrescentam detalhe de alta frequência.
- **PNG** suporta entrelaçamento Adam7 com até 7 passagens, de píxel em píxel (não linha a linha), permitindo uma renderização progressiva bidimensional mais uniforme.

4.2 Luz e Percepção da Cor

4.2.1 O espectro electromagnético e a luz visível

A cor é uma percepção subjectiva provocada pela radiação electromagnética que o sistema visual humano consegue detectar. Esta faixa visível corresponde a comprimentos de onda entre aproximadamente **380 nm** (violeta) e **700 nm** (vermelho). Abaixo dos 380 nm encontra-se a radiação ultravioleta; acima dos 700 nm, o infravermelho — ambas invisíveis ao olho humano.

No interior da janela visível, cada comprimento de onda corresponde perceptualmente a uma cor: o violeta e o azul nos comprimentos mais curtos, o verde e o amarelo nas frequências intermédias, e o laranja e o vermelho nos comprimentos mais longos. Esta relação entre comprimento de onda e cor é a base física de todos os modelos de cor digitais.

4.2.2 Anatomia da retina e fotorreceptores

A percepção da cor ocorre na **retina**, camada fotossensível do olho. A retina contém dois tipos de fotorreceptores com funções distintas:

- **Cones** — aproximadamente 6 a 7 milhões, concentrados na **fóvea** (zona central da retina). São os responsáveis pela visão cromática e funcionam bem em condições de iluminação adequada.
- **Bastonetes** — aproximadamente 115 a 120 milhões, distribuídos pela retina periférica. Respondem a baixas intensidades luminosas mas não distinguem cores — produzem apenas informação de luminância.

A concentração de cones na fóvea explica por que razão a percepção de cor é mais nítida quando se olha directamente para um objecto, e mais pobre na visão periférica.

4.2.3 Sensibilidade espectral dos cones S, M, L

Existem três tipos de cones, classificados pela sua sensibilidade espectral máxima:

Tipo	Designação	Pico (nm)	Cor percepcionada
S	Short /	440 nm	Azul
M	Medium /	535 nm	Verde
L	Large /	565 nm	Vermelho

A percepção da cor não decorre da activação de um único tipo de cone, mas da **combinação das respostas** dos três tipos. É esta tricromacia — o facto de existirem três canais de cor independentes — que fundamenta os modelos de cor RGB utilizados em todos os sistemas digitais de visualização.

4.2.4 Absorção relativa e construção da percepção cromática

A resposta de cada tipo de cone a uma determinada luz pode exprimir-se como percentagem de energia absorvida. Tome-se o exemplo da cor cian a 500 nm: o cone B (S) absorve 35% da energia, o cone G (M) absorve 15% e o cone R (L) apenas 2%. É a combinação destas três respostas que o cérebro interpreta como a cor cian — não um comprimento de onda isolado, mas um padrão de activação dos três canais.

Esta propriedade tem uma consequência fundamental para a tecnologia: é possível provocar qualquer percepção cromática misturando apenas três luzes (vermelha, verde e azul), desde que as suas intensidades relativas produzam o mesmo padrão de activação dos cones que a cor pretendida. É exactamente este princípio que os monitores de computador exploram.

4.2.5 Síntese aditiva e síntese subtrativa

A composição de cores pode seguir dois princípios opostos, consoante o suporte físico:

Síntese aditiva — aplica-se à luz emitida. Quando se combinam feixes de luz de diferentes cores, as suas energias somam-se: vermelho + verde + azul (R + G + B) produz branco; a ausência de luz produz preto. É o princípio dos ecrãs, monitores e projectores.

Síntese subtrativa — aplica-se a pigmentos e tintas. Um pigmento absorve determinados comprimentos de onda e reflecte os restantes. A combinação de cian (C), magenta (M) e amarelo (Y) absorve toda a luz visível, produzindo preto (teórico); a ausência de pigmento — o papel em branco — reflecte toda a luz. É o princípio da impressão.

	Síntese Aditiva	Síntese Subtrativa
Suporte	Luz emitida	Pigmento/tinta
Cores primárias	R, G, B	C, M, Y
Soma total	Branco	Preto (teórico)
Ausência	Preto	Branco
Aplicação	Ecrãs, monitores	Impressão

4.3 Modelos de Cor

4.3.1 Modelo RGB

O **RGB** (*Red, Green, Blue*) é o modelo de cor padrão para todos os sistemas digitais de visualização. Cada cor é descrita por três componentes — vermelho, verde e azul — cada uma com 8 bits (256 valores possíveis, de 0 a 255).

Com três canais de 8 bits cada, o número total de cores representáveis é:

$$2^8 \times 2^8 \times 2^8 = 256^3 = 16\,777\,216 \approx 16{,}7 \text{ milhões de cores}$$

Este modo designa-se **True Color** (24 bpp). O valor 0 num canal significa ausência dessa componente; o valor 255 representa a intensidade máxima. Assim, (255, 0, 0) é vermelho puro, (0, 255, 0) é verde puro, (255, 255, 255) é branco e (0, 0, 0) é preto.

4.3.2 Notação hexadecimal

Na web, as cores RGB são frequentemente expressas em notação hexadecimal, no formato `#RRGGBB`, onde cada par de dígitos hexadecimais representa um canal (de `00` a `FF`).

A conversão é directa: cada par hexadecimal corresponde a um valor decimal entre 0 e 255. Por exemplo, `#FF6600` representa R=255 (FF), G=102 (66), B=0 (00) — o que corresponde à cor laranja.

Existe também uma forma abreviada `#RGB`, válida quando cada par repete o mesmo dígito: `#F60` é equivalente a `#FF6600`.

4.3.3 Representação de cor em CSS

O CSS oferece quatro notações para especificar cores, cada uma com vantagens em contextos distintos:

Nomes de cor — o CSS define 147 nomes predefinidos (`red`, `blue`, `coral`, `steelblue`, etc.). São convenientes para prototipagem rápida mas limitados para design rigoroso.

```
color: steelblue;
background-color: coral;
```

Função `rgb()` — especifica os três canais com valores decimais de 0 a 255. É a representação mais directa do modelo RGB.

```
/* R=51, G=102, B=204 - azul médio */
color: rgb(51, 102, 204);
```

Função `hsl()` — especifica a cor por tom (*hue*, 0–360°), saturação (0–100%) e claridade (*lightness*, 0–100%). É mais intuitiva para ajustar cores em design: para escurecer uma cor, basta reduzir a claridade sem alterar o tom.

```
/* Tom 220° (azul), saturação 60%, claridade 50% */
color: hsl(220, 60%, 50%);
```

Canal alfa — `rgba()` e `hsla()` — acrescentam um quarto parâmetro de transparência, de 0 (totalmente transparente) a 1 (totalmente opaco).

```
/* Azul a 50% de opacidade */
color: rgba(51, 102, 204, 0.5);

/* Equivalente em HSL */
color: hsla(220, 60%, 50%, 0.5);
```

Notação	Código CSS	Resultado
Nome	`color: steelblue`	steelblue
rgb()	`rgb(51, 102, 204)`	#3366CC
hsl()	`hsl(220, 60%, 50%)`	hsl(220,60%,50%)
rgba()	`rgba(51, 102, 204, 0.5)`	50% opaco

4.3.4 Modelo CMYK

O **CMYK** (*Cyan, Magenta, Yellow, Key/Black*) é o modelo de cor utilizado em impressão. Baseia-se na síntese subtrativa: cada tinta absorve determinados comprimentos de onda e reflecte os restantes. O quarto canal — o preto (K, de *Key*) — é acrescentado por razões práticas: a combinação teórica C + M + Y deveria produzir preto, mas na prática produz um castanho escuro impuro, além de consumir mais tinta e secar mais lentamente.

Em contexto web, o CMYK não é utilizado directamente — os browsers trabalham exclusivamente com RGB. O conhecimento do modelo CMYK é relevante para trabalho gráfico destinado a impressão (cartazes, brochuras, embalagens), onde os ficheiros devem ser preparados em espaço CMYK para correspondência fiel com as tintas físicas.

4.3.5 Modelos HSV e HSL

Os modelos **HSV** (*Hue, Saturation, Value*) e **HSL** (*Hue, Saturation, Lightness*) descrevem a cor segundo três dimensões intuitivas:

- **Hue (tom)** — o ângulo no círculo cromático (0° a 360°): 0°/360° é vermelho, 120° é verde, 240° é azul.
- **Saturation (saturação)** — a vivacidade da cor (0% a 100%): 0% produz cinzento, 100% a cor mais viva possível.
- **Value / Lightness (valor/claridade)** — em HSV, 0% é sempre preto e 100% é a cor mais viva; em HSL, 0% é preto, 50% é a cor plena e 100% é branco.

A diferença prática entre HSV e HSL reside no comportamento da claridade máxima: em HSV, o valor 100% produz a cor saturada mais brilhante; em HSL, a claridade 100% produz sempre branco.

O CSS suporta directamente `hsl()` e `hsla()`, tornando o HSL o modelo mais prático para design web: ajustar o tom cria paletas harmoniosas, ajustar a saturação cria variantes neutras, e ajustar a claridade cria versões claras ou escuras da mesma cor.

4.4 Profundidade de Cor e Canal Alfa

4.4.1 Profundidade de cor (bit depth)

A **profundidade de cor** (*bit depth*) define quantos bits são utilizados para representar a cor de cada píxel. Quanto maior a profundidade, mais níveis de cor são possíveis — e mais fiel é a representação do original — mas também maior o tamanho do ficheiro.

Profundidade	Cores representáveis	Modo	Aplicação típica
1 bpp	2	Monocromático	Texto digitalizado, fax
8 bpp	256	Indexado	GIF, paletas limitadas
24 bpp	16 777 216	True Color	Padrão web — 3 canais × 8 bits
32 bpp	16 777 216 + alfa	True Color + alfa	PNG, WebP com transparência

O modo **True Color** a 24 bpp é o padrão de facto para a web: três canais de 8 bits (R, G, B) produzem mais de 16 milhões de cores, suficientes para representar fotografias e gradientes sem bandas perceptíveis de cor (*colour banding*).

4.4.2 Canal alfa e transparência

As imagens de **32 bits** acrescentam um quarto canal de 8 bits ao espaço RGB — o **canal alfa** — que controla a transparência de cada píxel individualmente:

$$32 \text{ bpp} = \underbrace{8\,\text{bits}}_{\text{R}} + \underbrace{8\,\text{bits}}_{\text{G}} + \underbrace{8\,\text{bits}}_{\text{B}} + \underbrace{8\,\text{bits}}_{\text{A (alfa)}}$$

O valor alfa 0 significa píxel totalmente transparente; o valor 255 significa totalmente opaco. Os valores intermédios produzem transparência parcial, o que permite composição (*compositing*) — sobrepor camadas com graus variáveis de opacidade.

Os formatos que suportam canal alfa são: **PNG** (alfa de 8 bits completo), **WebP** (alfa completo), e **GIF** (transparência binária — alfa = 0 ou 255, sem valores intermédios).

4.5 Resolução e Densidade de Píxeis

4.5.1 Resolução da imagem, PPI e DPI

O conceito de resolução tem significados distintos consoante o contexto:

Resolução da imagem — exprime-se em píxeis (largura × altura) e descreve o número total de elementos de imagem armazenados. É um atributo intrínseco do ficheiro, independente do suporte de saída.

PPI (*Pixels Per Inch*) — densidade de píxeis num ecrã, calculada dividindo o número de píxeis pelo tamanho físico em polegadas. Um ecrã Retina da Apple tem PPI 200, o que torna os píxeis individuais imperceptíveis à distância normal de visualização.

DPI (*Dots Per Inch*) — resolução de impressão, expressa em pontos de tinta por polegada. Para impressão de qualidade, o padrão é 300 DPI; para visualização em ecrã, o valor histórico de 72 DPI deriva dos primeiros monitores Macintosh. Na web, o DPI de um ficheiro de imagem é irrelevante — apenas a resolução em píxeis determina a qualidade visual.

4.5.2 Cálculo de PPI — exemplos reais

O PPI calcula-se dividindo o número de píxeis pelo comprimento físico correspondente em polegadas (1 polegada = 2,54 cm):

$$\text{PPI} = \frac{\text{n.º de píxeis}}{\text{comprimento em polegadas}}$$

Monitor Full HD (1920 × 1080) em diagonal de 30” (27” × 17”):

$$\text{PPI}_{\text{largura}} = \frac{1920}{27} \approx 71 \quad \text{PPI}_{\text{altura}} = \frac{1080}{17} \approx 64$$

Monitor Full HD (1920 × 1080) em diagonal de 65” (55” × 34,4”):

$$\text{PPI}_{\text{largura}} = \frac{1920}{55} \approx 35 \quad \text{PPI}_{\text{altura}} = \frac{1080}{34{,}4} \approx 31$$

TV UHD 4K (3840 × 2160) em diagonal de 65” (55” × 34,4”):

$$\mathrm{PPI}_{\text{largura}} = \frac{3840}{55} \approx 70 \quad \mathrm{PPI}_{\text{altura}} = \frac{2160}{34{,}4} \approx 63$$

Este último exemplo é revelador: apesar de a TV 4K ter exactamente o dobro dos píxeis do monitor Full HD de 30”, o PPI é semelhante. A densidade visual percepcionada depende não apenas da resolução, mas também do tamanho físico do ecrã — um monitor de alta resolução em ecrã grande pode ter densidade inferior à de um telemóvel com resolução menor.

4.6 Factor Gama e Correcção de Gama

4.6.1 Relação não-linear entre sinal e luminância

Os monitores e outros dispositivos de visualização apresentam uma relação não-linear entre o sinal de entrada (o valor numérico do píxel) e a luminância emitida (a quantidade de luz percepcionada). Esta relação é modelada pela equação:

$$L = V^{\gamma}$$

onde L é a luminância normalizada (0 a 1), V é o valor do sinal normalizado (0 a 1) e γ é o expoente de gama do dispositivo. Para os monitores comuns, $\gamma \approx 2{,}2$.

Esta não-linearidade significa que, sem correcção, uma imagem captada com intensidades lineares seria reproduzida de forma distorcida — as sombras ficariam demasiado escuras e os meios-tons deslocados.

4.6.2 Correcção de gama e o padrão sRGB

A solução adoptada universalmente é aplicar a correcção de gama em dois momentos complementares:

1. **Na aquisição (câmara)** — aplica-se o expoente inverso $1/\gamma \approx 1/2{,}2$ ao sinal captado. Esta operação designa-se *gamma encoding* ou *gamma correction*.
2. **Na reprodução (monitor)** — o monitor aplica $\gamma \approx 2{,}2$ ao sinal recebido.

Como os dois expoentes se cancelam ($\frac{1}{2{,}2} \times 2{,}2 = 1$), a imagem chega ao utilizador com as luminâncias correctas, independentemente das não-linearidades do dispositivo.

Se uma câmara filmar com $\gamma = 1$ (sinal linear, sem correcção), o monitor aplicará o seu $\gamma > 1$ sem compensação, e a imagem reproduzida terá luminâncias erradas — mais escura do que o original. Se a câmara codificar com $\gamma \approx 2{,}2$, o monitor cancela exactamente essa codificação e a imagem é reproduzida de forma fiel.

O padrão **sRGB** — o espaço de cor padrão da web e do CSS — define $\gamma \approx 2{,}2$ como expoente de referência. Todos os browsers assumem sRGB por omissão, o que garante consistência cromática entre dispositivos diferentes.

Gama e percepção humana

A correcção de gama tem também uma vantagem perceptiva: o olho humano é mais sensível a variações de luminância em tons escuros do que em tons claros. A codificação com $1/\gamma$ aloca mais níveis de quantização às sombras, onde o olho é mais discriminativo — o que melhora a eficiência da representação numérica da imagem.

4.7 Síntese do Capítulo

Uma imagem digital rasterizada é uma grelha de píxeis, cada um com um valor numérico de cor. O tamanho teórico de um ficheiro bitmap não comprimido obtém-se pelo produto largura × altura × bpp. As imagens vetoriais (SVG) descrevem formas por equações matemáticas, escalam sem perda de qualidade e são o formato adequado para logótipos e ícones na web.

A cor percepcionada pelo ser humano resulta da activação combinada de três tipos de cones na retina (S/azul, M/verde, L/vermelho), o que fundamenta a representação tricromática RGB adoptada em todos os sistemas digitais de visualização. A síntese aditiva (luz, RGB) e a síntese subtrativa (pigmento, CMYK) seguem princípios opostos e aplicam-se a contextos distintos — ecrãs e impressão, respectivamente.

O modelo RGB codifica cada cor em três canais de 8 bits (0–255), produzindo 24 bpp e 16,7 milhões de cores possíveis. O CSS suporta as notações hexadecimal (`#RRGGBB`), `rgb()`, `hsl()`, `rgba()` e `hsla()`. Os modelos HSV e HSL descrevem a cor de forma intuitiva (tom, saturação, brilho/claridade) e são mais práticos para design.

A profundidade de cor determina o número de níveis representáveis; o canal alfa (4.º canal de 8 bits) controla a transparência píxel a píxel nos modos de 32 bpp. A resolução de uma imagem para a web mede-se em píxeis; o PPI exprime a densidade num ecrã e depende tanto da resolução como do tamanho físico. O factor gama modela a relação não-linear entre sinal e luminância; o padrão sRGB (2,2) garante reprodução cromática consistente entre dispositivos.

4.8 Exercícios

Exercício 4.1 — Tamanho de ficheiro bitmap

Calcule o tamanho teórico (sem compressão) de cada imagem, em megabytes:

a) Imagem 800 × 600 píxeis a 24 bpp
b) Imagem 3840 × 2160 píxeis (4K) a 32 bpp
c) Imagem 320 × 240 píxeis a 8 bpp (paleta indexada)

Verificação: 1,37 MB; 31,64 MB; 0,073 MB.

Exercício 4.2 — Conversão de cor

Converta cada cor entre as notações indicadas:

a) `#3399FF` → valores decimais R, G, B

b) R=204, G=0, B=102 → notação hexadecimal
c) `hsl(0, 100%, 50%)` — que cor é esta? Qual o valor RGB equivalente?

Verificação: R=51, G=153, B=255; `#CC0066`; vermelho puro — R=255, G=0, B=0.

Exercício 4.3 — Escolha de formato de imagem

Para cada situação, indique o formato mais adequado e justifique:

a) Fotografia de produto com fundo branco para uma loja online
b) Logótipo em duas cores para o cabeçalho de um site, necessário em vários tamanhos
c) Ícone de interface com transparência para sobreposição sobre diferentes fundos
d) Animação simples de cinco fotogramas para uma barra de carregamento

Exercício 4.4 — Cálculo de PPI

Um ecrã de telemóvel tem resolução 2340 × 1080 píxeis e diagonal física de 6,1 polegadas. Calcule o PPI (use a diagonal em píxeis dividida pela diagonal física) e classifique a densidade obtida.
Verificação: diagonal em px 2580 px; PPI 423 — densidade Retina.

Exercício 4.5 — Canal alfa e cor em CSS

Escreva o código CSS para cada requisito:

a) Fundo azul royal (`#4169E1`) a 70% de opacidade, usando `rgba()`
b) O mesmo fundo especificado em `hsla()` (H 225°, S 73%, L 57%)
c) Um título com cor `tomato` convertida para `rgb()` (`rgb(255, 99, 71)`)
d) Um parágrafo com `hsl(120, 40%, 35%)` — que tonalidade é esta?

5 Layout CSS — Box Model, Flexbox e Grid

Introdução

O layout de uma página web é o processo pelo qual os elementos HTML são posicionados no espaço visual disponível. Durante décadas, este processo foi dominado por técnicas improvisadas — tabelas HTML, floats — que nunca foram concebidas para o propósito. O CSS moderno oferece hoje dois sistemas de layout dedicados: o **Flexbox** e o **CSS Grid**, cada um optimizado para um conjunto específico de problemas.

Neste capítulo estuda-se o box model — o modelo fundamental que governa o espaço ocupado por qualquer elemento HTML — a grelha manual de 12 colunas construída com floats, e os dois sistemas modernos de layout: Flexbox para distribuição unidimensional e Grid para composição bidimensional.

5.1 O Box Model

5.1.1 Definição e camadas

Cada elemento HTML é representado pelo browser como um rectângulo com quatro camadas concêntricas — o **box model**. Compreender estas camadas é condição prévia para qualquer trabalho de layout em CSS.

As quatro camadas, da mais interna para a mais externa, são:

- **Content** — a área onde o conteúdo real reside: texto, imagem, vídeo. As propriedades `width` e `height` definem as dimensões desta camada (no modo `content-box`, o padrão histórico do CSS).
- **Padding** — espaço interior entre o conteúdo e a fronteira. Tem a cor de fundo do elemento e aumenta a área clicável sem afectar a posição do conteúdo.
- **Border** — contorno do elemento. Tem espessura (`border-width`), estilo (`border-style`) e cor (`border-color`) configuráveis.
- **Margin** — espaço exterior entre o elemento e os seus vizinhos. As margens são transparentes — não herdam a cor de fundo do elemento.

5.1.2 box-sizing: content-box vs border-box

O comportamento histórico do CSS — definido na propriedade `box-sizing: content-box` — determina que `width` e `height` se aplicam **apenas à área de conteúdo**. O padding e a border são adicionados por fora, aumentando as dimensões totais do elemento. Este comportamento é sistematicamente contra-intuitivo:

```
/* content-box: largura total = 200 + 20 + 20 + 2 + 2 = 244px */
.caixa {
  width: 200px;
  padding: 20px;
  border: 2px solid #333;
  box-sizing: content-box; /* comportamento por omissão */
}
```

O modo `border-box` inclui o padding e a border dentro das dimensões declaradas — o que corresponde à intuição natural de "esta caixa tem 200px de largura":

```
/* border-box: largura total = 200px (exacamente) */
.caixa {
  width: 200px;
  padding: 20px;
  border: 2px solid #333;
  box-sizing: border-box; /* recomendado */
}
```

A prática moderna é aplicar `border-box` globalmente com o selector universal:

```
*, *::before, *::after {
  box-sizing: border-box;
}
```

Esta regra está presente em todos os frameworks CSS modernos, incluindo o Bootstrap.

> **Por que o content-box existe?**
>
> O `content-box` é o comportamento original do CSS 1 (1996), adoptado por razões de compatibilidade com os browsers da época. O Internet Explorer 6 e anteriores implementavam `border-box` por omissão, criando uma incompatibilidade histórica. O CSS 3 resolveu o problema tornando ambos os modos explicitamente configuráveis.

5.1.3 Margin, padding e border em CSS

As quatro camadas do box model são controladas por propriedades CSS com sintaxe uniforme. Todas suportam a notação shorthand — um único valor aplica-se a todos os lados; dois valores aplicam-se a {cima/baixo, esquerda/direita}; quatro valores aplicam-se a {cima, direita, baixo, esquerda} (sentido horário).

```
/* margin - shorthand */
.el { margin: 16px; }              /* todos os lados */
.el { margin: 8px 16px; }          /* cima/baixo | esquerda/direita */
.el { margin: 4px 8px 12px 6px; }  /* cima | direita | baixo | esquerda */
```

```
/* padding - mesma lógica */
.el { padding: 24px; }
.el { padding: 12px 24px; }

/* border */
.el { border: 1px solid #333; }
.el { border-top: 2px dashed #c00; border-bottom: none; }

/* centrar horizontalmente com margin auto */
.contentor {
  width: 960px;
  margin: 0 auto; /* margens laterais automáticas = centrado */
}
```

5.1.4 Colapso de margens

As margens verticais entre elementos de bloco adjacentes **colapsam**: em vez de se somarem, prevalece a maior das duas. Este comportamento designa-se *margin collapsing* e é uma fonte frequente de confusão.

```
/* Dois parágrafos consecutivos: margin-bottom 24px + margin-top 16px */
/* Espaço real entre eles: max(24, 16) = 24px - não 40px */
p { margin-bottom: 24px; }
h2 { margin-top: 16px; }
```

O colapso de margens não ocorre em contextos flex nem em contextos grid — apenas entre blocos no fluxo normal do documento.

5.2 Float e Grelha Manual

5.2.1 O float e o seu propósito original

A propriedade `float` foi concebida para um propósito editorial específico: permitir que texto flua em torno de uma imagem, à semelhança do comportamento de revistas impressas. Os valores possíveis são `left`, `right` e `none`.

```
/* Imagem com texto a fluir à direita */
img.destaque {
  float: left;
  margin: 0 16px 8px 0;
}
```

Durante os anos 2000, na ausência de alternativas nativas de layout, os programadores adaptaram o float para construir layouts multi-coluna — um uso para o qual nunca foi desenhado, com consequências que veremos a seguir.

5.2.2 O colapso do contentor e o clearfix

Um elemento flutuante **sai do fluxo normal** do documento. O resultado prático é que o seu contentor pai não contabiliza a sua altura — o contentor colapsa como se o elemento flutuante não existisse:

```
/* Problema: .pai não "vê" .filho flutuante */
.pai   { background: #eee; } /* altura = 0, mesmo com filho */
.filho { float: left; width: 200px; height: 150px; }
```

A solução clássica é o **clearfix** — uma técnica que força o contentor a englobar os seus filhos flutuantes através de um pseudo-elemento vazio:

```
/* Clearfix moderno com ::after */
.clearfix::after {
  content: "";
  display: table;
  clear: both;
}
```

Aplicando `.clearfix` ao contentor, este passa a incluir a altura dos seus filhos flutuantes.

5.2.3 Grelha de 12 colunas com floats

O sistema de 12 colunas é o layout mais utilizado no design web. A escolha de 12 justifica-se pela sua divisibilidade: admite colunas de 1, 2, 3, 4, 6 e 12 partes iguais, cobrindo a maioria dos layouts editoriais.

Numa implementação manual com floats, cada coluna é uma percentagem da largura do contentor. Com um *gutter* (espaço entre colunas) de 2%, cada coluna tem largura $(100\% - 11 \times 2\%)/12 \approx 6{,}5\%$. Na prática simplifica-se omitindo os gutters e usando padding interno:

```
/* Contentor */
.row { width: 100%; }
.row::after { content: ""; display: table; clear: both; }

/* Colunas - percentagens de 12 */
[class^="col-"] { float: left; }
.col-1  { width:  8.333%; }
.col-2  { width: 16.666%; }
.col-3  { width: 25%; }
.col-4  { width: 33.333%; }
.col-6  { width: 50%; }
.col-8  { width: 66.666%; }
.col-9  { width: 75%; }
.col-12 { width: 100%; }
```

Uso em HTML:

```
<div class="row">
  <div class="col-8">Conteúdo principal</div>
  <div class="col-4">Sidebar</div>
</div>
```

As colunas de uma linha devem sempre somar 12. Se a soma for inferior a 12, sobra espaço; se for superior, a última coluna salta para a linha seguinte.

5.2.4 Limitações do float como sistema de layout

Apesar da sua ubiquidade durante mais de uma década, o float apresenta limitações estruturais quando usado para layout:

- **Saída do fluxo** — os elementos flutuantes não participam no cálculo de altura do contentor, exigindo clearfix em cada linha.
- **Alinhamento vertical impossível** — centrar verticalmente dois elementos de alturas diferentes é impraticável com floats.
- **Reordenação visual** — a ordem visual é determinada pela ordem no HTML; reordenar visualmente exige alterar o markup.
- **Responsividade complexa** — adaptar uma grelha float a múltiplos breakpoints exige muitas media queries.

Estas limitações motivaram o desenvolvimento do Flexbox (2009–2012) e do CSS Grid (2017), que resolvem os problemas de layout de forma nativa.

5.3 Flexbox

5.3.1 O modelo flex

O **Flexbox** (*Flexible Box Layout*) é um modelo de layout unidimensional — distribui elementos ao longo de **uma** direcção: linha ou coluna. É optimizado para:

- Distribuir espaço entre elementos ao longo de um eixo
- Alinhar elementos relativamente a um eixo
- Adaptar automaticamente o tamanho dos elementos ao espaço disponível

O modelo assenta na distinção entre **contentor flex** (*flex container*) e **filhos flex** (*flex items*):

```
/* Activar flex no contentor */
.contentor { display: flex; }

/* Todos os filhos directos tornam-se automaticamente flex items */
```

> **Unidimensional vs bidimensional**
>
> O Flexbox é **unidimensional**: trabalha numa direcção de cada vez (linha *ou* coluna). O CSS Grid é **bidimensional**: trabalha em linha *e* coluna simultaneamente. Esta diferença determina qual dos dois usar em cada situação.

5.3.2 Eixo principal e eixo transversal

O Flexbox organiza-se em torno de dois eixos perpendiculares:

- **Eixo principal** (*main axis*) — a direcção em que os filhos são distribuídos. Definido por `flex-direction`.
- **Eixo transversal** (*cross axis*) — perpendicular ao eixo principal.

```
.contentor {
  display: flex;
  flex-direction: row;    /* padrão: esquerda → direita */
  /* flex-direction: row-reverse; - direita → esquerda */
  /* flex-direction: column;      - cima → baixo */
  /* flex-direction: column-reverse; - baixo → cima */
}
```

Quando `flex-direction: row`, o eixo principal é horizontal e o transversal é vertical. Quando `flex-direction: column`, invertem-se os papéis.

5.3.3 Propriedades do contentor flex

As propriedades que controlam o comportamento do contentor afectam todos os filhos em conjunto:

```
.contentor {
  display: flex;

  /* Direcção e quebra */
  flex-direction: row;           /* row | row-reverse | column | column-reverse */
  flex-wrap: nowrap;             /* nowrap | wrap | wrap-reverse */
  flex-flow: row nowrap;         /* shorthand: direction + wrap */

  /* Distribuição no eixo principal */
  justify-content: flex-start;  /* flex-start | flex-end | center |
                                   space-between | space-around | space-evenly */

  /* Alinhamento no eixo transversal */
  align-items: stretch;         /* stretch | flex-start | flex-end |
                                   center | baseline */

  /* Alinhamento de múltiplas linhas */
```

```
  align-content: flex-start;    /* (só quando flex-wrap: wrap) */

  /* Espaço entre filhos */
  gap: 16px;                    /* equivalente a row-gap + column-gap */
}
```

5.3.4 Propriedades dos filhos flex

Cada filho pode ter propriedades individuais que sobrepõem ou complementam o comportamento definido pelo contentor:

```
.filho {
  /* Crescimento, encolhimento e tamanho base */
  flex-grow:   0;    /* factor de crescimento (0 = não cresce) */
  flex-shrink: 1;    /* factor de encolhimento (1 = encolhe proporcionalmente) */
  flex-basis:  auto; /* tamanho base antes da distribuição de espaço */
  flex: 1;           /* shorthand: grow=1, shrink=1, basis=0 */

  /* Alinhamento individual (sobrepõe align-items do contentor) */
  align-self: center; /* auto | flex-start | flex-end | center | stretch */

  /* Ordem visual (sem alterar o HTML) */
  order: 0;          /* valor inteiro; menor = aparece primeiro */
}
```

A propriedade `flex: 1` é a mais usada na prática: aplica `flex-grow: 1`, `flex-shrink: 1` e `flex-basis: 0`, o que faz o filho ocupar uma quota igual do espaço disponível.

5.3.5 Casos de uso típicos do Flexbox

```
/* Centrar vertical e horizontalmente */
.contentor {
  display: flex;
  justify-content: center;
  align-items: center;
}

/* Navbar com logo à esquerda e links à direita */
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
}

/* Linha de cards com alturas iguais */
```

```
.cards {
  display: flex;
  gap: 16px;
}
.card {
  flex: 1;           /* distribuição igual */
  display: flex;
  flex-direction: column;
}
.card-footer {
  margin-top: auto;  /* empurra o rodapé para o fundo do card */
}

/* Layout principal com sidebar */
.layout {
  display: flex;
  gap: 24px;
}
.conteudo { flex: 3; }
.sidebar  { flex: 1; }
```

5.4 CSS Grid

5.4.1 O modelo de grelha bidimensional

O **CSS Grid** é um sistema de layout bidimensional — posiciona elementos ao longo de **dois eixos simultâneos**: linhas e colunas. É optimizado para:

- Layouts de página completos (header, nav, main, sidebar, footer)
- Grelhas de conteúdo regulares (galeria de imagens, lista de produtos)
- Posicionamento preciso de elementos em relação a uma grelha

```
/* Activar grid no contentor */
.contentor { display: grid; }
```

O conceito central é a **grelha explícita** — o programador declara quantas colunas e linhas existem, e o browser coloca os filhos segundo essas regras.

5.4.2 Definir colunas e linhas

As colunas e linhas são definidas pelas propriedades `grid-template-columns` e `grid-template-rows`:

```
.grelha {
  display: grid;

  /* 3 colunas: 200px fixo, 1fr flexível, 1fr flexível */
  grid-template-columns: 200px 1fr 1fr;

  /* 2 linhas automáticas */
  grid-template-rows: auto;

  /* Espaço entre células */
  gap: 16px;
}

/* A unidade fr (fraction) distribui o espaço disponível proporcionalmente */
/* 1fr + 2fr + 1fr = 4 partes: 25% + 50% + 25% */
.grelha-flex {
  display: grid;
  grid-template-columns: 1fr 2fr 1fr;
  gap: 12px;
}
```

A função `repeat()` evita repetição para grelhas regulares:

```
/* 12 colunas iguais */
.grelha-12 {
  display: grid;
  grid-template-columns: repeat(12, 1fr);
  gap: 16px;
}

/* Colunas responsivas automáticas: mínimo 200px, máximo 1fr */
.grelha-auto {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(200px, 1fr));
  gap: 16px;
}
```

5.4.3 Posicionamento de elementos

Por omissão, o CSS Grid coloca os filhos sequencialmente, célula a célula. Para controlo preciso, podem-se usar as propriedades de posicionamento:

```
/* Notação: linha-início / linha-fim */
.item-a {
  grid-column: 1 / 3;   /* ocupa colunas 1 e 2 */
  grid-row: 1 / 2;      /* ocupa a linha 1 */
}
```

```
/* Alternativa com span: ocupa N colunas a partir da posição actual */
.item-b {
  grid-column: span 2;  /* ocupa 2 colunas */
}

/* Posicionamento absoluto na grelha */
.destaque {
  grid-column: 2 / 4;
  grid-row: 1 / 3;
}
```

5.4.4 Template areas

A propriedade `grid-template-areas` permite nomear zonas da grelha com texto, tornando o layout visualmente declarativo:

```
.pagina {
  display: grid;
  grid-template-columns: 220px 1fr;
  grid-template-rows: auto 1fr auto;
  grid-template-areas:
    "header  header"
    "sidebar main"
    "footer  footer";
  min-height: 100vh;
  gap: 0;
}

.header  { grid-area: header; }
.sidebar { grid-area: sidebar; }
.main    { grid-area: main; }
.footer  { grid-area: footer; }
```

O mapa textual de `grid-template-areas` reproduz visualmente o layout pretendido. Um ponto (`.`) representa uma célula vazia.

5.4.5 Flexbox vs CSS Grid — quando usar cada um

A escolha entre Flexbox e Grid não é de exclusão mútua — os dois sistemas coexistem e complementam-se numa mesma página:

	Flexbox	CSS Grid
Dimensionalidade	Unidimensional (1 eixo)	Bidimensional (2 eixos)
Controlo	A partir do **filho** (tamanho flexível)	A partir do **contentor** (grelha explícita)

	Flexbox	CSS Grid
Caso de uso ideal	Distribuição de items em linha/coluna	Layouts de página, grelhas regulares
Alinhamento	Excelente num eixo	Excelente nos dois eixos
Reordenação	`order` por item	Posicionamento total
Suporte browsers	Excelente (IE 11+)	Excelente (Chrome 57+, 2017)

Regra prática:

- Usar **Flexbox** para componentes — navbar, cards em linha, botões alinhados, centrar um elemento.
- Usar **Grid** para a estrutura da página — header/main/sidebar/footer, galerias de imagens, layouts complexos.

```
/* Exemplo realista: Grid para a página, Flex para os componentes */

/* Estrutura da página - Grid */
.pagina {
  display: grid;
  grid-template-areas:
    "header"
    "main"
    "footer";
  grid-template-rows: auto 1fr auto;
  min-height: 100vh;
}

/* Navbar dentro do header - Flex */
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 0 24px;
}

/* Grelha de cards dentro do main - Grid */
.galeria {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(280px, 1fr));
  gap: 24px;
  padding: 24px;
}

/* Conteúdo interno de cada card - Flex */
.card {
  display: flex;
  flex-direction: column;
```

```
}
.card-footer { margin-top: auto; }
```

5.5 Síntese do Capítulo

O **box model** é o fundamento de todo o layout CSS: cada elemento ocupa um rectângulo com quatro camadas — conteúdo, padding, border e margin. A propriedade `box-sizing: border-box` torna o comportamento intuitivo e é adoptada por todos os frameworks modernos.

O **float** foi o mecanismo de layout dominante durante a primeira década da web responsiva. A grelha manual de 12 colunas, construída com floats e percentagens, demonstra os princípios que o Bootstrap depois automatizou. As suas limitações — colapso do contentor, impossibilidade de alinhamento vertical, complexidade responsiva — motivaram os sistemas modernos.

O **Flexbox** resolve o layout unidimensional: distribui elementos ao longo de uma linha ou coluna, alinha-os nos dois eixos e adapta os seus tamanhos ao espaço disponível. É o sistema adequado para componentes — navbars, filas de cards, centração de elementos.

O **CSS Grid** resolve o layout bidimensional: posiciona elementos em relação a uma grelha de linhas e colunas definida explicitamente no contentor. É o sistema adequado para a estrutura da página e para grelhas regulares. Os dois sistemas não são concorrentes — coexistem e complementam-se num mesmo projecto.

5.6 Exercícios

Exercício 5.1 — Box model e box-sizing

Dado o seguinte CSS em modo `content-box`:

```
.caixa {
  width: 300px;
  padding: 20px;
  border: 3px solid #333;
  margin: 16px;
}
```

a) Qual é a largura total ocupada pelo elemento (incluindo border)?
b) Qual seria a largura total em modo `border-box`?
c) Escreva a regra global que activa `border-box` em todos os elementos.

Verificação: a) 346px; b) 300px; c) `*, *::before, *::after { box-sizing: border-box; }`

Exercício 5.2 — Grelha de 12 colunas com floats

Construa, apenas com CSS e a técnica de floats, um layout com:

- Uma linha com três colunas de igual largura (4 + 4 + 4 colunas)
- Uma linha com uma coluna de conteúdo principal e uma sidebar (8 + 4 colunas)

Inclua o clearfix necessário para que o contentor não colapse.

Exercício 5.3 — Flexbox

Usando apenas Flexbox (sem Grid, sem floats), implemente:

a) Um contentor que centra um único elemento filho horizontal e verticalmente.
b) Uma barra de navegação com o logótipo à esquerda e quatro links à direita, todos alinhados verticalmente ao centro.
c) Uma linha de quatro cards que ocupa toda a largura disponível, com espaço igual entre os cards mas sem espaço nas extremidades.

Exercício 5.4 — CSS Grid

Usando CSS Grid e `grid-template-areas`, implemente o seguinte layout de página:

```
+-----------------------+
|        header         |
+-----------+-----------+
|  sidebar  |   main    |
+-----------+-----------+
|        footer         |
+-----------------------+
```

Condições: header e footer com altura fixa de 60px; sidebar com largura fixa de 240px; main ocupa o espaço restante; a página ocupa sempre 100% da altura do viewport.

Exercício 5.5 — Escolha do sistema de layout

Para cada situação, indique se usaria **Flexbox**, **Grid** ou **ambos**, e justifique:

a) Distribuir cinco ícones de redes sociais em linha, com espaçamento igual.
b) Construir o layout completo de uma página com header, dois sidebars e área central.
c) Criar uma galeria fotográfica responsiva com colunas que se adaptam automaticamente.
d) Alinhar verticalmente o texto e o ícone dentro de um botão.

6 Frameworks CSS, Bootstrap e Micro-interações

Introdução

O capítulo anterior construiu layouts a partir do zero — box model, floats, Flexbox, Grid. Este capítulo parte dessas fundações e introduz uma camada de abstração: os **frameworks CSS**, bibliotecas que encapsulam padrões de layout e componentes em classes reutilizáveis.

O Bootstrap é o framework mais utilizado no mundo e o escolhido nesta UC. Mas antes de o usar, é essencial compreender o que ele faz por baixo — a grelha de 12 colunas que estudámos no capítulo anterior, as media queries, os modelos Flexbox — porque essa compreensão é o que distingue quem usa um framework com domínio de quem apenas copia classes.

A segunda metade do capítulo aborda as **micro-interações** em CSS puro: transições, transformações e animações. O Bootstrap usa estas técnicas internamente; aprenderem-nas em CSS nativo significa levá-las para qualquer projecto, com ou sem framework.

6.1 O Problema que os Frameworks Resolvem

6.1.1 CSS manual a escala

Num projecto pequeno, escrever CSS do zero é rápido e directo. A partir de um certo ponto, a escala impõe três problemas estruturais:

Consistência — manter os mesmos espaçamentos, tamanhos e cores em dezenas de componentes (botões, formulários, navbars) exige repetição manual. Uma alteração ao estilo de um botão pode implicar editar centenas de linhas em múltiplos ficheiros.

Responsividade — escrever `@media` queries para todos os breakpoints de cada componente é trabalhoso. Vinte componentes com quatro breakpoints cada equivale a oitenta blocos de media queries para manter.

Manutenção — o CSS não tem sistema de tipos nem módulos nativos. CSS não estruturado cresce para um estado que os programadores designam por *CSS soup*: um ficheiro difícil de ler, difícil de alterar sem quebrar algo inesperado.

6.1.2 O que é um framework CSS

Um framework CSS é uma biblioteca de estilos, componentes e convenções prontas a usar — escritas uma vez, reutilizáveis em qualquer projecto. Em vez de escrever o CSS de cada botão ou navbar do zero, o framework fornece classes que se aplicam directamente no HTML.

O compromisso é claro: **ganha-se velocidade e consistência**, perde-se algum controlo fino sobre o CSS gerado. Na maioria dos projectos, este compromisso é favorável.

6.1.3 Panorama dos frameworks principais

Framework	Abordagem	Ponto forte	Caso de uso típico
Bootstrap	Componentes	Curva baixa, comunidade enorme	Projectos académicos, prototipagem, portais
Tailwind CSS	Utility-first	Controlo total, performance	Design systems, aplicações modernas
Foundation	Componentes	Grid avançado, SASS nativo	Projectos enterprise
Bulma	Componentes	CSS puro (sem JS), leve	Projectos ligeiros
Materialize	Material Design	Animações incluídas, estética Google	Apps com look Android/Google

> **Por que Bootstrap nesta UC?**
>
> Bootstrap combina curva de aprendizagem baixa, documentação exemplar e presença massiva no mercado de trabalho. É o ponto de entrada mais eficaz para um primeiro contacto com frameworks. Os conceitos aprendidos — grelha, breakpoints, utilitários — transferem-se para qualquer outro framework.

6.2 Bootstrap — Instalação via CDN

6.2.1 O que é um CDN

Um **CDN** (*Content Delivery Network*) é uma rede distribuída de servidores que disponibiliza ficheiros estáticos (CSS, JavaScript, fontes) a partir do nó geograficamente mais próximo do utilizador. Usar o Bootstrap via CDN significa não descarregar nem hospedar os ficheiros — o browser carrega-os directamente dos servidores do Bootstrap.

Vantagens:

- Configuração imediata — sem instalação, sem npm, sem build step
- O ficheiro pode estar em cache no browser do utilizador (se já visitou outro site com o mesmo CDN)
- Sempre actualizado para a versão especificada

Desvantagens:

- Dependência de conectividade — não funciona offline

- Impossível personalizar (alterar variáveis SASS, etc.)
- Produção: preferível servir os ficheiros do próprio servidor

6.2.2 Estrutura mínima de uma página Bootstrap

Uma página HTML com Bootstrap requer três elementos essenciais:

```html
<!DOCTYPE html>
<html lang="pt">
<head>
  <meta charset="UTF-8">

  <!-- 1. viewport meta - essencial para responsividade -->
  <meta name="viewport" content="width=device-width, initial-scale=1">

  <!-- 2. Bootstrap CSS - deve vir antes de qualquer CSS personalizado -->
  <link
    href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
    rel="stylesheet">

  <title>Página com Bootstrap</title>
</head>
<body>

  <!-- conteúdo aqui -->
  <div class="container">
    <h1>Olá, Bootstrap!</h1>
  </div>

  <!-- 3. Bootstrap JS Bundle - no final do body, antes de </body> -->
  <!-- inclui Popper.js para tooltips, dropdowns e modals -->
  <script
    src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js">
  </script>

</body>
</html>
```

> **CSS antes do JS**
>
> O CSS deve ser incluído no `<head>` para que o estilo seja aplicado antes de o conteúdo ser renderizado. O JS é colocado no final do `<body>` para não bloquear o carregamento da página. O `bootstrap.bundle.min.js` inclui já o Popper.js — não é necessário adicioná-lo separadamente.

6.2.3 CSS personalizado sobre Bootstrap

Ao adicionar CSS personalizado, este deve ser incluído **depois** do CSS do Bootstrap para que as regras próprias possam sobrepor as do framework:

```
<head>
  <!-- 1. Bootstrap CSS -->
  <link href="bootstrap.min.css" rel="stylesheet">

  <!-- 2. CSS próprio - vem depois, tem precedência -->
  <link href="styles.css" rel="stylesheet">
</head>
```

6.3 O Sistema de Grid Bootstrap

6.3.1 Container, row e col

O grid Bootstrap assenta em três classes hierárquicas que espelham exactamente a grelha de 12 colunas estudada no capítulo anterior — a diferença é que o Bootstrap gere o clearfix, os gutters e a responsividade automaticamente.

```
<div class="container">          <!-- centra e define largura máxima -->
  <div class="row g-3">          <!-- linha de grelha; g-3 = gutter de 1rem -->
    <div class="col-8">          <!-- ocupa 8 das 12 colunas -->
      Conteúdo principal
    </div>
    <div class="col-4">          <!-- ocupa 4 das 12 colunas -->
      Sidebar
    </div>
  </div>
</div>
```

As colunas da mesma `row` devem somar 12. A classe `col` (sem número) distribui o espaço restante igualmente entre todos os `col` da linha.

6.3.2 Os 6 breakpoints do Bootstrap

O Bootstrap define seis pontos de quebra, todos com `min-width` — confirmando que o Bootstrap é **mobile-first por definição**. Abaixo de 576px não existe media query; o CSS base aplica-se a todos os tamanhos.

Nome	Prefixo	Condição	Container
xs	*(nenhum)*	< 576px — sem query	100%
sm	`sm`	576px	540px
md	`md`	768px	720px
lg	`lg`	992px	960px
xl	`xl`	1200px	1140px
xxl	`xxl`	1400px	1320px

> **Container Breakpoint**
>
> O container é sempre mais estreito que o breakpoint — para `md` (768px), o container tem 720px. Os 48px restantes (24px de cada lado) formam as margens automáticas que centram o conteúdo. O `container-fluid` é a excepção: ocupa sempre 100% da largura.

6.3.3 Classes responsivas — col-{bp}-{n}

A responsividade Bootstrap é **declarativa**: especifica-se no HTML como cada coluna se comporta a cada breakpoint, sem escrever uma única media query:

```
<!-- Uma coluna que se adapta automaticamente:
     mobile (< 768px): linha completa
     tablet (  768px): meia largura
     desktop (  992px): um terço da largura -->

<div class="row g-3">
  <div class="col-12 col-md-6 col-lg-4">Card A</div>
  <div class="col-12 col-md-6 col-lg-4">Card B</div>
  <div class="col-12 col-md-6 col-lg-4">Card C</div>
</div>
```

As classes aplicam-se de forma **cumulativa** — `col-md-6` aplica-se em md *e em todos os breakpoints maiores*, até ser sobreposta por `col-lg-4`.

6.4 Componentes Bootstrap

6.4.1 Navbar

A navbar Bootstrap é responsiva por natureza: colapsa em mobile (mostrando o botão hambúrguer) e expande a partir do breakpoint indicado em `navbar-expand-{bp}`.

```
<nav class="navbar navbar-expand-lg bg-dark navbar-dark">
  <div class="container">

    <a class="navbar-brand" href="#">FAM</a>

    <!-- Botão hambúrguer - visível abaixo de lg (< 992px) -->
    <button class="navbar-toggler"
            data-bs-toggle="collapse"
            data-bs-target="#menu">
      <span class="navbar-toggler-icon"></span>
    </button>

    <!-- Área colapsável com os links -->
    <div class="collapse navbar-collapse" id="menu">
```

```
      <ul class="navbar-nav ms-auto">
        <li class="nav-item">
          <a class="nav-link active" href="#">Início</a>
        </li>
        <li class="nav-item">
          <a class="nav-link" href="#">Portfolio</a>
        </li>
        <li class="nav-item">
          <a class="nav-link" href="#">Contacto</a>
        </li>
      </ul>
    </div>
  </div>
</nav>
```

6.4.2 Cards

O card é o componente de conteúdo mais versátil do Bootstrap — combina imagem, texto e acção num bloco visual consistente.

```
<div class="card" style="width: 18rem;">
  <img src="foto.jpg" class="card-img-top" alt="Imagem do projecto">
  <div class="card-body">
    <h5 class="card-title">Projecto Web</h5>
    <p class="card-text">Descrição breve do projecto desenvolvido.</p>
    <a href="#" class="btn btn-primary">Ver mais</a>
  </div>
  <div class="card-footer text-muted">
    Actualizado há 2 dias
  </div>
</div>
```

6.4.3 Buttons

Os botões Bootstrap seguem o padrão `btn` + `btn-{variante}`, com variantes sólidas e de contorno (*outline*), e três tamanhos.

```
<!-- Variantes sólidas -->
<button class="btn btn-primary">Primary</button>
<button class="btn btn-secondary">Secondary</button>
<button class="btn btn-success">Success</button>
<button class="btn btn-danger">Danger</button>
<button class="btn btn-warning">Warning</button>

<!-- Variantes outline -->
<button class="btn btn-outline-primary">Outline Primary</button>
```

```
<!-- Tamanhos -->
<button class="btn btn-primary btn-lg">Grande</button>
<button class="btn btn-primary">Normal</button>
<button class="btn btn-primary btn-sm">Pequeno</button>

<!-- Largura total -->
<button class="btn btn-primary w-100">Bloco (100%)</button>
```

6.4.4 Forms

```
<form>
  <div class="mb-3">
    <label for="email" class="form-label">Endereço de email</label>
    <input type="email" class="form-control" id="email"
           placeholder="nome@exemplo.pt">
  </div>
  <div class="mb-3">
    <label for="msg" class="form-label">Mensagem</label>
    <textarea class="form-control" id="msg" rows="3"></textarea>
  </div>
  <div class="mb-3 form-check">
    <input type="checkbox" class="form-check-input" id="aceito">
    <label class="form-check-label" for="aceito">Aceito os termos</label>
  </div>
  <button type="submit" class="btn btn-primary w-100">Enviar</button>
</form>
```

6.5 Utilitários Bootstrap

Os utilitários são classes atómicas que substituem CSS de uma linha — aplicadas directamente no HTML, sem escrever regras adicionais.

6.5.1 Spacing — margin e padding

O padrão é `{tipo}{lado}-{escala}`, onde o tipo é `m` (margin) ou `p` (padding), o lado é opcional (`t`, `b`, `s`, `e`, `x`, `y`), e a escala vai de 0 a 5 (em múltiplos de `0.25rem`) ou `auto`.

```
<div class="mt-3">   <!-- margin-top: 1rem -->
<div class="mb-2">   <!-- margin-bottom: 0.5rem -->
<div class="px-4">   <!-- padding horizontal: 1.5rem -->
<div class="py-2">   <!-- padding vertical: 0.5rem -->
<div class="m-0">    <!-- margin: 0 em todos os lados -->
<div class="mx-auto"> <!-- margin horizontal: auto → centrado -->
```

Escala	Valor rem	Valor px (base 16px)
0	0	0
1	0.25rem	4px
2	0.5rem	8px
3	1rem	16px
4	1.5rem	24px
5	3rem	48px

6.5.2 Display, Flexbox e texto

```
<!-- Display -->
<div class="d-none">    <!-- display: none -->
<div class="d-flex">    <!-- display: flex -->
<div class="d-md-none"> <!-- none em  md, visível abaixo -->

<!-- Flexbox (quando d-flex está activo) -->
<div class="d-flex justify-content-between align-items-center">
<div class="d-flex gap-3 flex-wrap">

<!-- Texto -->
<p class="text-center">  <!-- text-align: center -->
<p class="fw-bold">      <!-- font-weight: bold -->
<p class="text-muted">   <!-- cor cinzento subtil -->
<p class="fs-4">         <!-- font-size nível 4 (1.5rem) -->

<!-- Cores de fundo e texto -->
<div class="bg-primary text-white">
<div class="bg-light text-dark">

<!-- Dimensões -->
<div class="w-100">      <!-- width: 100% -->
<div class="h-100">      <!-- height: 100% -->
```

6.6 Micro-interações em CSS Puro

6.6.1 O Bootstrap já anima — mas como?

Se inspeccionarmos o CSS do Bootstrap com as DevTools do browser (**F12** → **Sources** → **bootstrap.min.css**), encontramos isto:

```
/* Bootstrap internals - transition nos botões */
.btn {
  transition: color .15s ease-in-out,
              background-color .15s ease-in-out,
              border-color .15s ease-in-out,
```

```
              box-shadow .15s ease-in-out;
}

/* Transição no collapse da navbar */
.collapsing { transition: height .35s ease; }

/* Fade nos modals */
.fade { transition: opacity .15s linear; }
```

O efeito de hover dos botões, a abertura suave da navbar, o fade dos modals — nada disto é JavaScript. É CSS puro. O Bootstrap escreve as transições por nós; as secções seguintes ensinam a fazê-lo por conta própria, em qualquer contexto, com ou sem framework.

6.6.2 transition

A propriedade `transition` suaviza a mudança de qualquer propriedade CSS ao longo do tempo. A sintaxe é:

```
transition: propriedade duração easing atraso;
```

```
/* Exemplo simples: mudar a cor de fundo em 0.3s */
.btn {
  background: #0d6efd;
  transition: background-color 0.3s ease;
}
.btn:hover { background: #0a58ca; }

/* Múltiplas propriedades */
.card {
  transform: translateY(0);
  box-shadow: none;
  transition: transform 0.2s ease,
              box-shadow 0.2s ease;
}
.card:hover {
  transform: translateY(-6px);
  box-shadow: 0 8px 24px rgba(0,0,0,.15);
}

/*  Evitar: transition: all - anima tudo, impacto de performance */
```

Os quatro parâmetros de `transition`:

Parâmetro	Valores	Descrição
`propriedade`	`color`, `opacity`, `transform`, …	O que muda. Preferir propriedades específicas a `all`.
`duração`	`0.3s`, `300ms`	Quanto tempo dura a transição.
`easing`	`ease`, `linear`, `ease-in`, `ease-out`, `cubic-bezier()`	Curva de velocidade.
`atraso`	`0s`, `0.1s`	Espera antes de começar. Opcional.

6.6.3 transform

A propriedade `transform` modifica a aparência visual de um elemento — movendo-o, escalando-o ou rodando-o — **sem alterar o fluxo do documento**. Os elementos vizinhos não se deslocam. O browser usa aceleração GPU, o que torna as animações com `transform` mais eficientes do que animar `top` ou `left`.

```
/* Mover */
.el:hover { transform: translateY(-8px); }   /* desloca para cima 8px */
.el:hover { transform: translateX(4px); }    /* desloca para a direita */
.el:hover { transform: translate(-4px, -4px); } /* ambos os eixos */

/* Escalar */
.el:hover { transform: scale(1.05); }        /* 5% maior */
.el:hover { transform: scale(0.95); }        /* 5% menor */

/* Rodar */
.arrow:hover { transform: rotate(90deg); }   /* sentido horário */
.icon.open   { transform: rotate(45deg); }   /* ícone × a partir de + */

/* Combinar - sempre numa única declaração */
.card:hover {
  transform: translateY(-4px) scale(1.02);   /*   */
}
/*  Errado: duas declarações transform sobrepoem-se */
.card:hover { transform: translateY(-4px); }
.card:hover { transform: scale(1.02); }      /* cancela a anterior */
```

6.6.4 Pseudo-classes como gatilhos

As pseudo-classes definem **estados** de um elemento — são o mecanismo que activa as micro-interações sem JavaScript.

```
.btn {
  background: #0d6efd;
```

```
  color: white;
  transition: background 0.2s ease, transform 0.1s ease;
}

/* Estado: rato sobre o elemento */
.btn:hover {
  background: #0a58ca;
  transform: translateY(-2px);
}

/* Estado: elemento pressionado */
.btn:active {
  transform: translateY(0);   /* "afunda" no clique */
}

/* Estado: elemento com foco (teclado ou clique) */
.btn:focus {
  outline: 3px solid rgba(13,110,253,.4);
  outline-offset: 2px;
}

/* Estado: foco apenas por teclado (não por clique) */
/* mais limpo que :focus para utilizadores de rato */
.btn:focus-visible {
  outline: 3px solid #0d6efd;
  outline-offset: 2px;
}
```

> **:focus e acessibilidade**
>
> Nunca remover o outline de foco sem fornecer uma alternativa visual. O outline é o único indicador de posição para utilizadores que navegam por teclado (pessoas com limitações motoras). O `:focus-visible` permite mostrar o outline apenas durante navegação por teclado, sem o mostrar a utilizadores de rato — o melhor dos dois mundos.

6.6.5 @keyframes — animações autónomas

Enquanto `transition` reage a uma mudança de estado (hover, focus), o `@keyframes` define uma sequência de estados que corre automaticamente, independentemente da interação do utilizador.

A utilização é um processo em dois passos:

Passo 1 — Definir a animação com `@keyframes`:

```
/* Sintaxe with from/to */
@keyframes fadeInUp {
  from {
```

```
    opacity: 0;
    transform: translateY(20px);
  }
  /* ponto intermédio opcional: */
  /* 50% { opacity: 0.5; }      */
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

/* Sintaxe com percentagens - permite múltiplos estados */
@keyframes pulse {
  0%, 100% { transform: scale(1); }
  50%       { transform: scale(1.05); }
}
```

Passo 2 — **Aplicar** com a propriedade `animation`:

```
/* Shorthand: nome duração easing atraso iterações direção fill-mode */
.hero    { animation: fadeInUp 0.6s ease-out; }
.badge   { animation: pulse 2s ease-in-out infinite; }

/* Propriedades individuais */
.card {
  animation-name:            fadeInUp;
  animation-duration:        0.4s;
  animation-timing-function: ease-out;
  animation-delay:           0.1s;
  animation-fill-mode:       forwards; /* mantém o estado final */
}
```

> **animation-fill-mode: forwards**
>
> Por omissão, quando uma animação termina, o elemento volta ao estado inicial. `fill-mode: forwards` mantém o estado final da animação — essencial para efeitos de entrada como `fadeInUp`, onde queremos que o elemento permaneça visível depois de aparecer.

6.6.6 Boas práticas e acessibilidade

```
/*  Especificar propriedades - não usar transition: all */
.btn { transition: background-color 0.2s ease, transform 0.1s ease; }

/*  Preferir transform e opacity - aceleração GPU, sem reflow */
/*  Evitar animar top, left, width, height - causam reflow */
```

```
/*  Respeitar prefers-reduced-motion */
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    animation-duration:   0.01ms !important;
    transition-duration:  0.01ms !important;
  }
}

/*  Nunca remover :focus sem alternativa visível */
/*  .btn:focus { outline: none; }  - inacessível */
/*  .btn:focus-visible { outline: 3px solid #86b7fe; } */

/*  Animar com propósito - cada micro-interação deve comunicar algo:
      hover → "isto é clicável"
      active → "o clique foi registado"
      fadeInUp → "novo conteúdo chegou"      */
```

6.7 Síntese do Capítulo

Os **frameworks CSS** resolvem o problema da consistência e da responsividade a escala. O Bootstrap é o mais adoptado: inclui via CDN, usa classes no HTML, e encapsula as media queries, o grid e os componentes mais comuns.

O **grid Bootstrap** é o sistema de 12 colunas do capítulo anterior, automatizado e responsivo. Os seis breakpoints (`xs` a `xxl`) usam todos `min-width` — o Bootstrap é mobile-first por definição. As classes `col-{bp}-{n}` aplicam-se a partir do breakpoint indicado e são cumulativas.

Os **componentes** (navbar, cards, buttons, forms) e os **utilitários** (spacing, display, flex, texto, cores) eliminam CSS repetitivo. A regra de ouro é incluir o CSS próprio **depois** do Bootstrap para poder sobrepor estilos.

As **micro-interações** — `transition`, `transform`, pseudo-classes e `@keyframes` — são CSS puro, independentes do Bootstrap. O Bootstrap usa-as internamente; aprendê-las em CSS nativo significa usá-las em qualquer projecto. `transform` e `opacity` são as propriedades a animar: usam GPU, não causam reflow, e produzem transições fluidas. Respeitar `prefers-reduced-motion` e nunca remover `:focus` sem alternativa são requisitos de acessibilidade, não opções.

6.8 Exercícios

Exercício 6.1 — Incluir Bootstrap e estrutura base

Crie uma página HTML mínima que:

a) Inclua Bootstrap 5.3 via CDN (CSS no `<head>`, JS no final do `<body>`).
b) Tenha o `<meta name="viewport">` correcto.
c) Mostre o texto "Olá, Bootstrap!" centrado horizontalmente com a classe `text-`

`center`, dentro de um `container`.

Verifique que a página funciona offline sem a CDN e observe o que muda.

Exercício 6.2 — Grid responsivo

Construa uma grelha Bootstrap com:

a) Três cards por linha em desktop (992px).
b) Dois cards por linha em tablet (768px e < 992px).
c) Um card por linha em mobile (< 768px).

Cada card deve ter título, parágrafo e um botão. Use `h-100` para garantir que os cards da mesma linha ficam com a mesma altura. Use `g-3` para espaçamento entre cards.

Exercício 6.3 — Navbar responsiva

Implemente uma navbar Bootstrap que:

a) Mostre o nome do site (`navbar-brand`) à esquerda.
b) Tenha quatro links de navegação à direita (`ms-auto`).
c) Colapse em mobile (abaixo de 992px) com o botão hambúrguer.
d) Use fundo escuro (`bg-dark navbar-dark`).

Teste o comportamento no browser redimensionando a janela.

Exercício 6.4 — transition e :hover

Crie um card com:

a) Transição de `transform` de 0.25s quando o rato passa por cima.
b) O card sobe 8px (`translateY(-8px)`) no hover.
c) Uma sombra suave (`box-shadow`) que aparece suavemente no hover.
d) O elemento deve ter `transition` definido no estado base, não no `:hover`.

Explique por que razão a `transition` deve ser definida no estado base e não no `:hover`.

Exercício 6.5 — @keyframes

Crie uma animação `slideInLeft` que:

a) Começa com `opacity: 0` e `transform: translateX(-30px)`.
b) Termina com `opacity: 1` e `transform: translateX(0)`.
c) Dura 0.5s com easing `ease-out`.
d) Aplica-se a um elemento `.hero` que aparece quando a página carrega.
e) Adicione a regra `prefers-reduced-motion` correspondente.

7 SASS Fundamental

Introdução

O capítulo anterior introduziu o Bootstrap — um *framework* CSS que encapsula grelhas, componentes e utilitários em classes prontas a usar. O que ficou por dizer é que o Bootstrap é, ele próprio, **escrito em SASS**. As variáveis que definem cores, tamanhos e *breakpoints* são variáveis SASS; os *mixins* que geram as classes responsivas do *grid* são *mixins* SASS. Quem domina SASS compreende o Bootstrap de dentro para fora.

Este capítulo introduz o SASS como linguagem independente — aplicável a qualquer projecto, com ou sem *framework*. O ponto de partida é o problema que o SASS resolve: o CSS, escrito sem estrutura, não escala.

7.1 O Problema do CSS a Escala

7.1.1 Valores repetidos

Num projecto real, a mesma cor primária aparece em dezenas de sítios: no botão, no *link*, no *header*, no destaque de formulário. Se o cliente pedir para alterar a cor, cada ocorrência tem de ser encontrada e editada manualmente:

```
/* Num projecto sem SASS - a mesma cor em seis lugares */
.btn              { background: #5f4fb8; }
.link             { color: #5f4fb8; }
.header           { border-bottom: 2px solid #5f4fb8; }
.badge            { background: #5f4fb8; }
.form-control:focus { border-color: #5f4fb8; }
.pagination .active { background: #5f4fb8; }
```

Um *find-and-replace* não é solução — pode atingir a mesma *string* noutros contextos. O SASS resolve isto com uma **variável**: altera-se num único lugar e a mudança propaga-se para todo o projecto.

7.1.2 Falta de estrutura

O HTML organiza-se hierarquicamente — um `.card` contém um `.card__titulo` que contém um `h3`. No CSS, essa hierarquia desaparece: cada selector é escrito de forma plana, sem relação visual com a estrutura HTML:

```
/* CSS plano - a hierarquia do HTML é invisível */
.card                      { background: white; border-radius: 8px; }
.card .card__titulo        { font-size: 1.25rem; font-weight: bold; }
.card .card__titulo:hover  { color: #5f4fb8; }
.card .card__corpo         { padding: 1rem; }
.card .card__footer        { font-size: 0.85rem; color: #666; }
```

À medida que o projecto cresce, este CSS disperso torna-se difícil de ler e de associar mentalmente à estrutura HTML correspondente.

7.1.3 Manutenção difícil

Sem módulos, um projecto CSS de dimensão média acaba num único ficheiro com centenas ou milhares de linhas. Alterar um componente implica saber onde começa e termina o seu CSS, num ficheiro sem divisões claras. O resultado é o que os programadores designam por *CSS soup*: funciona, mas ninguém quer tocar.

7.2 O que é o SASS

7.2.1 Pré-processador CSS

O SASS (*Syntactically Awesome Style Sheets*) é um **pré-processador CSS** — uma linguagem que estende o CSS com variáveis, *nesting*, *mixins*, funções e módulos, e que **compila** para CSS válido antes de chegar ao *browser*. Isto é, desenvolve-se o projecto em SASS e há um processo automático (compilador) que gera o CSS.

```
Ficheiro .scss  →  Compilador SASS  →  Ficheiro .css  →  Browser
```

Um *browser* não funciona com SASS, mas apenas com CSS. O *browser* nunca vê código SASS. O que é servido é CSS puro — exactamente o mesmo que seria escrito à mão, mas gerado a partir de uma fonte mais estruturada e mais fácil de manter.

> **Dart Sass — a implementação actual**
>
> A implementação oficial e activa do SASS é o **Dart Sass**. Versões anteriores (LibSass, Ruby Sass) estão descontinuadas. O comando `sass` no terminal e a extensão Live Sass Compiler do VS Code usam Dart Sass.

7.2.2 Sintaxe SCSS

O SASS tem duas sintaxes. A mais usada é a **SCSS** (ficheiros `.scss`), que é um *superset* do CSS — qualquer CSS válido é também SCSS válido. A sintaxe alternativa `.sass` usa indentação em vez de chavetas, mas raramente é adoptada em projectos novos.

```
/* Isto é SCSS válido */
.btn {
  background: #5f4fb8;
  color: white;
  padding: 0.5rem 1rem;

  &:hover {
    background: #4a3d9a;  /* nesting - ver secção Nesting */
  }
}
```

O compilador transforma o bloco acima no CSS equivalente:

```
/* CSS compilado */
.btn { background: #5f4fb8; color: white; padding: 0.5rem 1rem; }
.btn:hover { background: #4a3d9a; }
```

7.2.3 Compilação no fluxo de trabalho

Em desenvolvimento, o SASS compila automaticamente a cada gravação de ficheiro. As ferramentas mais comuns:

Ferramenta	Como funciona
VS Code — Live Sass Compiler	Extensão que monitoriza `.scss` e compila para `.css` a cada gravação
CLI Dart Sass	`sass --watch scss/main.scss css/main.css` via terminal
WebStorm — File Watchers	Configuração nativa que executa o compilador automaticamente

7.3 Variáveis

7.3.1 Declaração e uso

Uma variável SASS guarda um valor que pode ser reutilizado em qualquer parte do projecto. As variáveis são escritas num ficheiro separado, geralmente com o nome `_variables.scss`. A sintaxe é `$nome: valor`:

```sass
/* Exemplo de variáveis definidas no ficheiro _variables.scss */
$cor-primaria:  #5f4fb8;
$cor-texto:     #1a1a1a;
$fonte-base:    'Segoe UI', Arial, sans-serif;
$tamanho-base:  16px;
$raio:          8px;
$espaco-md:     1rem;

/* As variáveis usam-se dentro de outros ficheiros. P. ex. em _components.scss */
.btn {
  background:    $cor-primaria;
  color:         white;
  font-family:   $fonte-base;
  font-size:     $tamanho-base;
  padding:       $espaco-md 2rem;
  border-radius: $raio;
}

.link { color: $cor-primaria; }
```

```sass
/* O CSS compilado fica assim: */
.btn {
  background: #5f4fb8; color: white;
  font-family: 'Segoe UI', Arial, sans-serif;
  font-size: 16px; padding: 1rem 2rem; border-radius: 8px;
}
.link { color: #5f4fb8; }
```

Para alterar a cor primária de todo o projecto, basta editar uma linha em `_variables.scss`.

7.3.2 Tipos de valores

As variáveis SASS aceitam qualquer valor CSS, e também tipos próprios:

Tipo	Exemplos
Cor	`#5f4fb8`, `rgb(95, 79, 184)`, `hsl(249, 40%, 51%)`
Número com unidade	`16px`, `1rem`, `100%`, `0.5s`
String	`'Segoe UI'`, `bold`, `center`
Booleano	`true`, `false`
Lista	`'Segoe UI', Arial, sans-serif`
Mapa	`(sm: 576px, md: 768px, lg: 992px)`
Null	`null` — a propriedade é omitida do *output*

7.3.3 Âmbito — global vs local

Uma variável declarada **fora** de qualquer bloco é global — acessível em todo o ficheiro. Declarada **dentro** de um bloco, é local — acessível apenas nesse bloco e nos seus descendentes:

```
$cor-primaria: #5f4fb8;   /* global */

.card {
  $raio-interno: 6px;      /* local - só existe dentro de .card */

  border-radius: $raio-interno;  /*  dentro do âmbito */
  color: $cor-primaria;          /*  variável global */
}

.btn {
  border-radius: $raio-interno;  /*  fora do âmbito - erro de compilação */
}
```

7.3.4 Variáveis SASS vs Custom Properties CSS

Pode surgir uma questão legítima: se o problema dos valores repetidos é tão evidente, o CSS não o terá resolvido por conta própria? A resposta é sim — o **CSS tem o seu próprio sistema de variáveis**, as *custom properties* (propriedades personalizadas), desenvolvido de forma completamente independente do SASS. Quem trabalha com SASS depara-se portanto com **dois mecanismos de variáveis distintos**, e convém perceber as diferenças.

As *custom properties* CSS escrevem-se com dois traços antes do nome e declaram-se habitualmente no selector `:root` (que corresponde ao elemento `<html>`, o topo da árvore da página), para ficarem disponíveis em todo o documento. Para ler o valor, usa-se a função `var()`:

```
/* Custom properties CSS - variáveis nativas da linguagem */
:root {
  --cor-primaria: #5f4fb8;
  --tamanho-fonte: 16px;
}

.btn {
  background: var(--cor-primaria);   /* acede-se ao valor com var() */
  font-size:  var(--tamanho-fonte);
}
```

À primeira vista, o resultado é semelhante ao das variáveis SASS: um valor centralizado, reutilizável em todo o documento. A diferença fundamental está em **quando** cada mecanismo actua:

- As variáveis SASS (`$var`) **desaparecem na compilação** — o compilador substitui cada `$cor-primaria` pelo valor concreto `#5f4fb8` antes de gerar o CSS. O *browser* nunca chega a ver a variável SASS.
- As *custom properties* CSS (`--var`) **persistem no browser** — o *browser* recebe o ficheiro CSS com `--cor-primaria: #5f4fb8` declarado e resolve a função `var()` em tempo de execução (à medida que a página é apresentada e pode até ser alterada enquanto a página está aberta).

Esta diferença tem consequências práticas:

Característica	SASS `$var`	CSS `--var`
Existe no CSS compilado?	Não	Sim
Acessível em JavaScript?	Não	Sim (`getComputedStyle`)
Pode mudar em tempo de execução?	Não	Sim (*media queries*, JS)
Funciona em `calc()`?	Não (usa operadores SASS)	Sim
Ideal para	Valores de design estáticos	Temas dinâmicos, adaptação responsiva

Usar os dois em conjunto

Uma boa prática é definir valores em variáveis SASS e expô-los como custom properties no `:root`, combinando o melhor de ambos:

```
$cor-primaria: #5f4fb8;

:root {
  --cor-primaria: #{$cor-primaria};  /* interpolação: #{} converte para string */
}
```

O JavaScript (a linguagem de programação dos *browsers*, que permite modificar a página depois de carregada) pode ler e alterar `--cor-primaria` em tempo de execução; o SASS usa `$cor-primaria` para cálculos durante a compilação.

7.4 Nesting

7.4.1 Hierarquia no CSS

O *nesting* permite escrever regras CSS aninhadas, espelhando a hierarquia do HTML. O compilador transforma cada nível aninhado num selector descendente:

```
/* SCSS */
.card {
  background: white;
  border-radius: $raio;

  .card__titulo {
    font-size: 1.25rem;
    font-weight: bold;
  }

  .card__corpo {
    padding: $espaco-md;
  }
}
```

```
/* CSS compilado */
.card { background: white; border-radius: 8px; }
.card .card__titulo { font-size: 1.25rem; font-weight: bold; }
.card .card__corpo  { padding: 1rem; }
```

A relação visual entre `.card` e os seus elementos é imediata no SCSS; no CSS plano equivalente, tem de ser inferida pelo leitor.

7.4.2 O `&` — referência ao selector pai

O `&` representa o selector do bloco pai. Permite escrever pseudo-classes, pseudo-elementos e modificadores no contexto do componente:

```
.btn {
  background: $cor-primaria;
  transition: background 0.2s ease, transform 0.1s ease;

  &:hover        { background: color.adjust($cor-primaria, $lightness: -10%); }
  &:active       { transform: translateY(1px); }
  &:focus-visible {
    outline: 3px solid $cor-primaria;
    outline-offset: 2px;
  }

  /* Modificadores BEM */
  &--grande   { font-size: 1.125rem; padding: 0.75rem 2rem; }
  &--contorno {
    background: transparent;
    border: 2px solid $cor-primaria;
    color: $cor-primaria;
  }
}
```

```
/* CSS compilado */
.btn { background: #5f4fb8; transition: background 0.2s ease, transform 0.1s ease; }
.btn:hover         { background: #4a3d9a; }
.btn:active        { transform: translateY(1px); }
.btn:focus-visible { outline: 3px solid #5f4fb8; outline-offset: 2px; }
.btn--grande       { font-size: 1.125rem; padding: 0.75rem 2rem; }
.btn--contorno     { background: transparent; border: 2px solid #5f4fb8; color: #5f4fb8;
```

7.4.3 A regra dos 3 níveis

O *nesting* em excesso gera selectores CSS excessivamente específicos e difíceis de sobrepor:

```
/*  Nesting excessivo - 5 níveis */
.header {
  .nav {
    .nav__lista {
      .nav__item {
        .nav__link { color: white; }
        /* compila para: .header .nav .nav__lista .nav__item .nav__link */
      }
    }
  }
}
```

A regra prática é **não ultrapassar 3 níveis de nesting**. Se o quarto nível for necessário, é geralmente sinal de que o componente deve ser refactorado em blocos independentes.

```
/*  Máximo 3 níveis */
.nav {
  &__lista { list-style: none; display: flex; }
  &__item  { margin: 0 0.5rem; }
  &__link  {
    color: white;
    &:hover { color: $cor-primaria; }  /* 3.º nível - aceitável */
  }
}
```

7.5 Partials e Módulos

7.5.1 Partials

Um **partial** é um ficheiro SCSS cujo nome começa com `_` (*underscore*). O compilador nunca o processa de forma autónoma — só o inclui quando é importado por outro ficheiro. Esta convenção permite dividir o CSS em ficheiros temáticos sem gerar um `.css` individual para cada um:

```
scss/
  _variables.scss   ← tokens de design (cores, tipografia, espaçamentos)
  _base.scss        ← reset e estilos globais
  _components.scss  ← botões, cards, mensagens
  _layout.scss      ← header, footer, grelha
  main.scss         ← ponto de entrada - importa todos os anteriores
```

Só o `main.scss` (sem *underscore*) compila para `css/main.css`. Os *partials* são peças que o `main.scss` agrega.

7.5.2 @import vs @use

Existem duas formas de incluir um *partial*: `@import` (método antigo, em extinção) e `@use` (método moderno, com *namespacing*):

```
/* main.scss - método @import */
@import 'variables';    /* todas as variáveis ficam globais */
@import 'base';
@import 'components';
@import 'layout';
```

```
/* _layout.scss - método @use para módulos nativos */
@use 'sass:color';      /* acesso via color.adjust() */
@use 'sass:math';       /* acesso via math.div() */
```

> **Por que @import está a ser abandonado?**
>
> O `@import` torna todas as variáveis, *mixins* e funções globais — num projecto grande, torna-se impossível saber de onde vem cada símbolo. O `@use` resolve isto com **namespacing**: `color.adjust()` indica explicitamente que a função vem do módulo `color`. O `@import` será removido no Dart Sass 3.0.

7.5.3 Módulos nativos do Dart Sass

O Dart Sass inclui módulos nativos que substituem funções globais em extinção:

```
@use 'sass:math';   /* math.div(), math.round(), math.floor(), math.ceil() */
@use 'sass:color';  /* color.adjust(), color.mix(), color.scale() */
@use 'sass:string'; /* string.length(), string.to-upper-case() */
@use 'sass:list';   /* list.length(), list.nth() */
```

7.6 Mixins

7.6.1 @mixin — bloco reutilizável

Um *mixin* é um bloco de CSS nomeado que pode ser incluído em qualquer selector. É a resposta do SASS ao problema da repetição de padrões:

```
/* Definição */
@mixin flex-center {
  display: flex;
  align-items: center;
  justify-content: center;
```

```
}

/* Uso com @include */
.hero            { @include flex-center; min-height: 100vh; }
.modal           { @include flex-center; position: fixed; inset: 0; }
.cartao__imagem  { @include flex-center; height: 160px; }
```

```
/* CSS compilado */
.hero           { display:flex; align-items:center; justify-content:center; min-height:10
.modal          { display:flex; align-items:center; justify-content:center; position:fixe
.cartao__imagem { display:flex; align-items:center; justify-content:center; height:160px;
```

7.6.2 Mixins com parâmetros

Os *mixins* aceitam parâmetros, tornando-os equivalentes a funções que geram CSS:

```
@use 'sass:color';

@mixin btn($cor-fundo: #5f4fb8, $cor-texto: white) {
  background:    $cor-fundo;
  color:         $cor-texto;
  padding:       0.5rem 1.5rem;
  border:        2px solid transparent;
  border-radius: 6px;
  cursor:        pointer;
  font-weight:   600;
  transition:    background 0.2s ease, transform 0.1s ease;

  &:hover  { background: color.adjust($cor-fundo, $lightness: -10%); }
  &:active { transform: translateY(1px); }
}

/* Com valores por defeito */
.btn-primario { @include btn; }

/* Sobrepondo parâmetros */
.btn-sucesso  { @include btn(#198754); }
.btn-perigo   { @include btn(#dc3545); }
.btn-escuro   { @include btn(#212529, white); border-color: #495057; }
```

7.6.3 Mixins com @content

O `@content` permite passar um bloco de CSS para dentro de um *mixin* — útil para encapsular *media queries* repetidas:

```
@mixin responde($bp) {
  @if $bp == sm { @media (min-width: 576px) { @content; } }
  @if $bp == md { @media (min-width: 768px) { @content; } }
  @if $bp == lg { @media (min-width: 992px) { @content; } }
}

.grelha {
  grid-template-columns: 1fr;

  @include responde(sm) { grid-template-columns: repeat(2, 1fr); }
  @include responde(md) { grid-template-columns: repeat(3, 1fr); }
}
```

```
/* CSS compilado */
.grelha { grid-template-columns: 1fr; }
@media (min-width: 576px) { .grelha { grid-template-columns: repeat(2, 1fr); } }
@media (min-width: 768px) { .grelha { grid-template-columns: repeat(3, 1fr); } }
```

7.7 Extend e Placeholders

7.7.1 @extend

O `@extend` faz com que um selector herde todos os estilos de outro. Em vez de duplicar propriedades, o compilador agrupa os selectores numa única regra:

```
.mensagem {
  padding: 1rem; border-radius: 6px;
  border-left: 4px solid; font-size: 0.9rem; margin-bottom: 1rem;
}

.mensagem-sucesso {
  @extend .mensagem;
  background: #d1e7dd; border-color: #198754; color: #0a3622;
}

.mensagem-erro {
  @extend .mensagem;
  background: #f8d7da; border-color: #dc3545; color: #58151c;
}
```

```
/* CSS compilado - selectores agrupados */
.mensagem, .mensagem-sucesso, .mensagem-erro {
  padding: 1rem; border-radius: 6px; border-left: 4px solid;
  font-size: 0.9rem; margin-bottom: 1rem;
}
```

```
.mensagem-sucesso { background: #d1e7dd; border-color: #198754; color: #0a3622; }
.mensagem-erro    { background: #f8d7da; border-color: #dc3545; color: #58151c; }
```

7.7.2 Placeholders %

Um **placeholder** é um selector especial que começa com `%`. Não gera CSS por si próprio — só existe para ser estendido. Resolve o problema de `@extend .mensagem` incluir a classe base no *output* mesmo que não seja usada directamente no HTML:

```
/* Placeholder - não gera CSS standalone */
%mensagem-base {
  padding: 1rem; border-radius: 6px;
  border-left: 4px solid; font-size: 0.9rem; margin-bottom: 1rem;
  line-height: 1.5;
}

.mensagem-sucesso {
  @extend %mensagem-base;
  background: #d1e7dd; border-color: #198754; color: #0a3622;
}

.mensagem-aviso {
  @extend %mensagem-base;
  background: #fff3cd; border-color: #ffc107; color: #664d03;
}
```

```
/* CSS compilado - sem .mensagem-base a mais */
.mensagem-sucesso, .mensagem-aviso {
  padding: 1rem; border-radius: 6px; border-left: 4px solid;
  font-size: 0.9rem; margin-bottom: 1rem; line-height: 1.5;
}
.mensagem-sucesso { background: #d1e7dd; border-color: #198754; color: #0a3622; }
.mensagem-aviso   { background: #fff3cd; border-color: #ffc107; color: #664d03; }
```

7.7.3 @extend vs @mixin — quando usar cada um

	`@extend` / `%placeholder`	`@mixin`
Aceita parâmetros?	Não	Sim
Output CSS	Agrupa selectores	Repete propriedades
Pode ser usado em *media queries*?	Não	Sim
Ideal para	Variantes de um base sem parâmetros	Padrões com variações configuráveis

> **Regra prática**
>
> Usar `%placeholder` + `@extend` quando as variantes partilham exactamente o mesmo bloco base e não há parâmetros. Usar `@mixin` quando há parâmetros ou quando o padrão é usado dentro de *media queries*.

7.8 Funções e Operadores

7.8.1 Operadores aritméticos

O SASS suporta operações aritméticas directamente nas propriedades CSS:

```
$tamanho-base: 16px;
$espaco-base:  1rem;

.titulo    { font-size: $tamanho-base * 2;   }  /* 32px  */
.subtitulo { font-size: $tamanho-base * 1.5; }  /* 24px  */
.espaco-lg { margin:    $espaco-base * 2;    }  /* 2rem  */
```

O operador `/` foi **removido como operador de divisão** no Dart Sass moderno, pois conflituava com a sintaxe CSS legítima `font: 14px/1.5`. A divisão é agora feita com `math.div()`:

```
@use 'sass:math';

$tamanho-base: 16px;
$espaco-base:  1rem;
$num-colunas:  3;

$tamanho-pequeno: math.div($tamanho-base, 1.25);  /* 12.8px   */
$espaco-sm:       math.div($espaco-base, 2);      /* 0.5rem   */
$largura-coluna:  math.div(100%, $num-colunas);   /* 33.333%  */
```

> **Por que o `/` foi removido?**
>
> Em CSS, `font: 14px/1.5` é sintaxe válida (tamanho/line-height). Para evitar ambiguidade, o Dart Sass removeu `/` como operador de divisão. O compilador emite um aviso de extinção se `/` for usado como divisão sem `math.div()`.

7.8.2 Funções de cor — color.adjust()

O módulo `sass:color` oferece funções para manipular cores de forma programática, eliminando a necessidade de calcular manualmente variantes mais claras ou escuras:

```
@use 'sass:color';

$cor-primaria: #5f4fb8;

.btn {
  background: $cor-primaria;

  &:hover {
    background: color.adjust($cor-primaria, $lightness: -10%);  /* mais escuro */
  }

  &:disabled {
    background: color.adjust($cor-primaria, $lightness: 30%);   /* desbotado */
  }
}

/* Outros usos comuns */
$cor-sombra: color.adjust($cor-primaria, $alpha: -0.7);         /* 30% opacidade */
$cor-borda:  color.adjust($cor-primaria, $saturation: -20%);    /* menos saturada */
```

7.8.3 Outras funções úteis

```
@use 'sass:math';
@use 'sass:color';

/* Arredondamento */
$col: math.round(math.div(100%, 3));  /* 33% */

/* Mistura de cores */
$meio: color.mix(#5f4fb8, white, 50%);  /* 50% primária + 50% branco */

/* color.scale() - ajuste relativo ao valor actual (mais previsível que adjust) */
$claro: color.scale($cor-primaria, $lightness: +40%);
$escuro: color.scale($cor-primaria, $lightness: -30%);
```

7.9 Exemplo Integrado

7.9.1 Estrutura do projecto

O projecto de demonstração da aula organiza-se em quatro *partials* mais um ponto de entrada. Cada ficheiro tem uma responsabilidade única:

```
scss/
   _variables.scss   ← variáveis globais (cores, tipografia, espaços, breakpoints)
   _base.scss        ← reset e estilos do body
```

```
_components.scss  ← botões (mixin) + mensagens (placeholder + extend)
_layout.scss      ← grelha responsiva, header, footer
main.scss         ← ponto de entrada - @import de todos os anteriores
```

7.9.2 _variables.scss — base de tudo {#sec-07-exemplo-variables}

```sass
@use 'sass:math';

/*   Cores                      */
$cor-primaria:   #ea9672;
$cor-secundaria: rgba(45, 27, 0, 0.635);
$cor-texto:      #1a1a1a;
$cor-fundo:      #f7f3ee;
$cor-branco:     #ffffff;

/*   Tipografia                   */
$fonte-base:      'Segoe UI', Arial, sans-serif;
$tamanho-base:    16px;
$tamanho-pequeno: math.div($tamanho-base, 1.25);  /* 12.8px */
$tamanho-grande:  $tamanho-base * 1.5;            /* 24px   */

/*   Espaçamentos                 */
$espaco-base: 1rem;
$espaco-sm:   math.div($espaco-base, 2);  /* 0.5rem */
$espaco-md:   $espaco-base;               /* 1rem   */
$espaco-lg:   $espaco-base * 2;           /* 2rem   */

/*   Breakpoints                  */
$bp-sm: 576px;
$bp-md: 768px;

/*   Outros                     */
$raio:   12px;
$sombra: 0 4px 12px rgba(0, 0, 0, 0.1);
```

Repare-se como `math.div()` é necessário para dividir valores com unidade: `16px / 1.25` deixou de ser divisão válida no Dart Sass moderno.

7.9.3 _components.scss — mixin e placeholder em acção {#sec-07-exemplo-components}

```sass
@use 'sass:color';

/*   Mixin de botão com parâmetros         */
@mixin btn($cor-fundo: $cor-primaria, $cor-txt: $cor-branco) {
```

```
  background:    $cor-fundo;
  color:         $cor-txt;
  padding:       $espaco-sm $espaco-lg;
  border:        2px solid transparent;
  border-radius: $raio;
  cursor:        pointer;
  font-size:     $tamanho-base;
  font-family:   $fonte-base;
  transition:    background 0.2s ease, transform 0.1s ease;

  &:hover  { background: color.adjust($cor-fundo, $lightness: -10%); }
  &:active {
    background: color.adjust($cor-fundo, $lightness: -18%);
    transform: translateY(1px);
  }
}

.btn-primario { @include btn($cor-primaria); }
.btn-escuro   { @include btn($cor-secundaria); }
.btn-contorno {
  @include btn($cor-branco, $cor-primaria);
  border-color: $cor-primaria;
  &:hover { background: color.adjust($cor-primaria, $lightness: 38%); }
}

/*   Placeholder para mensagens          */
%mensagem-base {
  padding:       $espaco-md;
  border-radius: $raio;
  border-left:   4px solid;
  font-size:     $tamanho-pequeno;
  margin-bottom: $espaco-md;
  line-height:   1.5;
}

.mensagem-sucesso {
  @extend %mensagem-base;
  background:   color.adjust(#28a745, $lightness: 48%);
  border-color: #28a745;
  color:        color.adjust(#28a745, $lightness: -12%);
}

.mensagem-erro {
  @extend %mensagem-base;
  background:   color.adjust(#dc3545, $lightness: 38%);
  border-color: #dc3545;
  color:        color.adjust(#dc3545, $lightness: -12%);
}
```

Neste ficheiro convergem três conceitos do capítulo: o *mixin* parametrizado gera três variantes

de botão a partir de uma única definição; o *placeholder* evita duplicação nas mensagens sem poluir o CSS com selectores desnecessários; o `color.adjust()` calcula automaticamente as variantes *hover* e as cores de estado.

7.9.4 _layout.scss — grelha responsiva {#sec-07-exemplo-layout}

```
@use 'sass:color';
@use 'sass:math';

$num-colunas:    3;
$largura-coluna: math.div(100%, $num-colunas);  /* 33.333% */

/* Grelha - mobile-first */
.grelha {
  display: grid;
  grid-template-columns: 1fr;      /* mobile: 1 coluna */
  gap: $espaco-lg;

  @media (min-width: $bp-sm) { grid-template-columns: repeat(2, 1fr); }
  @media (min-width: $bp-md) { grid-template-columns: repeat($num-colunas, 1fr); }
}

/* Header */
.header {
  background: $cor-secundaria;
  padding: $espaco-md $espaco-lg;

  &__interior {
    max-width: 1100px; margin: 0 auto;
    display: flex; justify-content: space-between; align-items: center;
  }

  &__nav {
    display: flex; gap: $espaco-lg; list-style: none;
    a {
      color: color.adjust($cor-primaria, $lightness: 20%);
      font-size: $tamanho-pequeno;
      transition: color 0.2s ease;
      &:hover { color: $cor-secundaria; }
    }
  }
}

/* Responsividade do header */
@media (max-width: $bp-sm) {
  .header__interior { flex-direction: column; align-items: flex-start; gap: $espaco-sm; }
  .header__nav      { flex-direction: column; gap: $espaco-sm; }
}
```

As *media queries* estão aninhadas dentro de `.grelha`, tirando partido do *nesting* SASS — a relação entre o componente e o seu comportamento responsivo é visualmente imediata, sem necessitar de um bloco `@media` separado no final do ficheiro.

7.10 Síntese do Capítulo

O **SASS** resolve três problemas estruturais do CSS: valores repetidos, falta de hierarquia visível e manutenção difícil. É um pré-processador — compila para CSS antes de chegar ao *browser*. O *browser* nunca vê código SASS.

As **variáveis** (`$nome: valor`) centralizam *tokens* de design. Alterar uma variável propaga a mudança para todo o projecto. Distinguem-se das CSS Custom Properties: as variáveis SASS desaparecem na compilação; as `--var` persistem no *browser* e podem ser alteradas em tempo de execução por JavaScript.

O **nesting** espelha a hierarquia do HTML no CSS. O `&` referencia o selector pai, permitindo escrever pseudo-classes, pseudo-elementos e modificadores BEM no contexto do componente. A regra dos 3 níveis evita selectores excessivamente específicos.

Os **partials** (`_ficheiro.scss`) dividem o CSS em módulos temáticos. Só o ponto de entrada (sem *underscore*) compila para `.css`. O `@use` é o método moderno de inclusão — com *namespacing*, substitui o `@import` em extinção.

Os **mixins** (`@mixin` / `@include`) encapsulam padrões CSS reutilizáveis com parâmetros. O **extend** e os **placeholders** (`%`) reutilizam blocos base sem parâmetros, agrupando selectores no CSS compilado.

As **funções** dos módulos `sass:math` e `sass:color` permitem cálculos e manipulações de cor em tempo de compilação. `math.div()` substitui o operador `/`, removido para evitar conflito com a sintaxe CSS de `font`.

7.11 Exercícios

Exercício 7.1 — Variáveis e estrutura base

Crie um projecto SASS com a estrutura mínima: `_variables.scss`, `_base.scss`, `main.scss`.

a) Em `_variables.scss`, defina: cor primária, cor de texto, tamanho base (16px), espaçamento base (1rem) e raio de bordo (8px).
b) Calcule com `math.div()`: tamanho pequeno (base ÷ 1.25) e espaçamento pequeno (base ÷ 2). Confirme os valores no CSS compilado.
c) Em `_base.scss`, aplique as variáveis ao `body` (font-size, color, font-family).
d) Em `main.scss`, importe os dois *partials* na ordem correcta.

Explique por que a ordem de importação dos *partials* importa.

Exercício 7.2 — Nesting e o `&`

Crie um componente `.cartao` com *nesting* SASS:

a) `.cartao` com fundo branco, raio de bordo e sombra.
b) `.cartao__titulo` e `.cartao__corpo` como elementos aninhados.
c) `.cartao:hover` com `transform: translateY(-4px)` e sombra mais intensa — usando `&`.
d) Não ultrapasse 3 níveis de *nesting*.

Escreva o CSS compilado equivalente à mão e compare com o *output* real do compilador.

Exercício 7.3 — Mixin de botão

Crie um *mixin* `btn` com dois parâmetros: `$cor-fundo` (por defeito: cor primária) e `$cor-texto` (por defeito: branco).

a) O *mixin* deve incluir: padding, border-radius, cursor e transição de background.
b) O estado `&:hover` deve escurecer `$cor-fundo` com `color.adjust(..., $lightness: -10%)`.
c) Crie três variantes: `.btn-primario`, `.btn-sucesso` (#198754) e `.btn-perigo` (#dc3545).
d) Adicione `.btn-contorno`: fundo transparente, texto e borda na cor primária.

Exercício 7.4 — Extend e placeholders

Crie um sistema de mensagens com três variantes (sucesso, aviso, erro):

a) Defina um *placeholder* `%alerta-base` com o estilo comum (padding, border-left, border-radius, font-size).
b) Crie `.alerta-sucesso`, `.alerta-aviso` e `.alerta-erro` usando `@extend %alerta-base`.
c) Cada variante deve ter cor de fundo, borda e texto distintas.
d) Examine o CSS compilado e explique por que o *placeholder* não aparece como selector no *output*.

Compare o *output* com o que seria gerado usando `@mixin` em vez de `@extend`.

Exercício 7.5 — Projecto completo com partials

Organize um mini-projecto com a estrutura completa de quatro *partials*:

a) `_variables.scss` — cores, tipografia, espaçamentos, *breakpoints* `$bp-sm: 576px` e `$bp-md: 768px`.
b) `_layout.scss` — `.grelha` com `@use 'sass:math'`, `$largura-coluna: math.div(100%, 3)`, e *media queries nested* para 1 → 2 → 3 colunas.
c) `_components.scss` — pelo menos um *mixin* e um *placeholder*, cada um reutilizado em dois selectores diferentes.
d) `main.scss` — importa os três *partials*.

Redimensione o *browser* para verificar o comportamento responsivo da grelha nos três

breakpoints.